M03 — Prácticas integrales: arquitectura hexagonal con .NET 8

Autor del curso: JAIME GALLO

Cómo utilizar el manual

El alumnado continúa el proyecto AceriaApp que culminó en **M02 2.4**. Cada apartado tiene 24 pasos y una carpeta de referencia autónoma. La práctica indica **dónde abrir o crear cada archivo, qué pegar o modificar, por qué, cómo compilar y qué comprobar**. El resultado no se obtiene copiando previamente una solución terminada.

Requisitos del puesto: Visual Studio con el componente ASP.NET/web, SDK .NET 8, SQL Server Express LocalDB para ejecutar la API en Windows; los tests HTTP usan SQL Server real con `ACERIA_TEST_SQLSERVER` o Docker y Testcontainers. Conserva `tests/Compartido/SqlServerPruebas.cs`, el enlace `Compile` y `Testcontainers.MsSql` 3.10.0 heredados de M02 2.1.22. La aplicación mantiene los endpoints de M02. En 3.3 aparecen proyectos de Dominio, Aplicación e Infraestructura con referencias físicas comprobables; en 3.4 el adaptador de ASP.NET Core y el adaptador EF Core utilizan un puerto real de repositorio.

Secuencia de trabajo: 3.1 capas y regla de dependencia → 3.2 puertos y adaptadores → 3.3 separación física de los proyectos → 3.4 registro de dependencias, repositorio SQL Server, endpoints HTTP y tests. Nunca mezcles el estado final de una etapa futura en el proyecto que estás construyendo.

Convención de código: cada bloque marcado con un archivo indica si se crea o se modifica. Si se trata de una modificación, localiza una sola vez el ancla indicada e inserta el código. No reemplaces un archivo heredado salvo cuando el bloque lo diga explícitamente.

Preparar Docker Desktop en Windows para los tests SQL (opcional)

La **API se ejecuta en Windows con SQL Server Express LocalDB**. Docker solo es necesario si eliges la variante de pruebas SQL Server Express mediante Testcontainers; **no se despliega aquí la aplicación entera**. Trabaja con PowerShell y realiza estas comprobaciones antes del primer test que necesite SQL Server:

1. Instala Docker Desktop para Windows usando el motor **WSL 2** y contenedores **Linux**; abre Docker Desktop y espera a que indique que el motor está iniciado. Si ya utilizas LocalDB, puedes omitir Docker. La instalación, las redes y los despliegues completos de contenedores se estudiarán por separado.
2. En PowerShell ejecuta `docker version` y `docker compose version`. Comprueba el tipo del servidor con `docker info --format '{{.OSType}}'`: debe devolver `linux`. Si da un error de conexión, inicia Docker Desktop; no continúes.
3. Comprueba el motor con `docker run --rm hello-world`. La primera vez debe descargarse una imagen; no es todavía la prueba de SQL Server. Verifica espacio disponible y memoria suficiente para SQL Server Express.
4. **Opción Docker/Testcontainers:** desde la raíz de una etapa **ya completada**, ejecuta `Remove-Item Env:\ACERIA_TEST_SQLSERVER -ErrorAction SilentlyContinue`, seguido de `dotnet test .\AceriaApp.sln -c Release` (en la primera etapa de M02, usa el `.csproj` de tests si aún no existe la solución). El helper crea un contenedor temporal SQL Server Express 2022, bases aisladas y las elimina. La imagen de SQL Server se descarga automáticamente; **no hace falta un `compose.yml` de SQL**.

5. **Alternativa LocalDB sin Docker:** ejecuta `sqllocaldb start MSSQLLocalDB`, configura `$env:ACERIA_TEST_SQLSERVER = 'Server=(localdb)\MSSQLLocalDB;Database=master;Integrated Security=True;TrustServerCertificate=True'` y ejecuta de nuevo los tests. Ambas vías usan SQL Server real; no se permite sustituirlo por SQLite o InMemory.
6. **Diagnóstico:** si falla Docker, comprueba que Docker Desktop está abierto y en modo Linux; si falla SQL, revisa la salida del test, la cadena de conexión a `master` y los permisos de creación de bases. No consideres una compilación como prueba de persistencia ni ignores pruebas fallidas.

Resultado exigido: `dotnet test` debe terminar con todas las pruebas de la etapa en estado correcto. El comando `docker ps -a` sirve para observar los contenedores temporales durante la ejecución, no para atribuir automáticamente un PASS. Esta preparación no altera el proyecto AceriaApp ni exige Linux como sistema operativo del alumno.

Controles acumulados previstos

Punto	Nuevos tests de catálogo	Tests adicionales de integración	Acumulado del curso vigente
Base M02 2.4	—	—	16
3.1	7	0	23
3.2	8	0	31
3.3	7	0	38
3.4	8	4	50

Punto 3.1 — Capas de la aplicación

Comprender la regla de dependencia antes de crear más proyectos

La etapa 3.1 conserva la API REST y su persistencia de M02. El objetivo ahora consiste en distinguir responsabilidades y explicar por qué el dominio no debería depender de ASP.NET Core o EF Core. La clasificación inicial de capas y el validador son herramientas docentes: prueban reglas declaradas, pero todavía no demuestran una separación física de ensamblados.

Trabajaremos sobre una copia del estado 2.4, para no perder los endpoints y las pruebas anteriores. Cada clase nueva se incorporará en el punto indicado y se comprobará mediante compilación y tests. La documentación y el ADR permitirán comparar las reglas que describimos con el código que construiremos en las etapas posteriores.

Objetivos del punto. el documento de capas de la aplicación es el artefacto de `docs` específico del punto 3.1. Se crea en este punto y conserva intactos los archivos heredados de M02 2.4.

Carpeta de trabajo: `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.1-capas-aplicacion`. Esta etapa continúa la anterior sin alterar su copia. El código de la referencia es un resultado final, no una solución que el alumno deba pegar al inicio.

Criterio de aceptación: seguir los 24 pasos, compilar y ejecutar los tests; comprobar los resultados indicados y registrar las actividades manuales. Al finalizar, la solución debe reproducir el comportamiento descrito.

PASO 3.1.1 — Crear la estructura del módulo 3 y del punto 3.1

Separar una copia para cada etapa permite comparar cambios sin dañar M02. El nuevo `global.json` fija la referencia del SDK; no hay que trasladar binarios compilados ni credenciales a la carpeta del alumno.

1. En el Explorador de archivos localiza `M02_APIS_REST_ASPNET_CORE/PROYECTO/2.4-documentacion-openapi`. Comprueba que es el estado anterior completo, con los endpoints y tests ya construidos.
2. Crea una carpeta personal `3.1-capas-aplicacion` bajo `M03_ARQUITECTURA_HEXAGONAL/PROYECTO`. Copia en ella el contenido del punto anterior, sin cambiarlo en su origen.
3. Desde la raíz de `3.1-capas-aplicacion` comprueba que tienes `src/AceriaApp.Api`, `tests/AceriaApp.Api.Tests`, `AceriaApp.http` y los archivos de configuración.
4. No copies `bin/`, `obj/` ni credenciales. Para comparar puedes usar los proyectos de referencia; los nuevos archivos de este punto se introducirán solo cuando llegues a su paso.

Archivo que debe quedar tras este paso: `global.json`

```
{
  "sdk": { "version": "8.0.400", "rollForward": "latestPatch" }
}
```

Verificación concreta. Desde la raíz del repositorio, ejecuta `dir M03_ARQUITECTURA_HEXAGONAL\PROYECTO\3.1-capas-aplicacion`. Deben aparecer `README.md`, `src`, `tests`, `AceriaApp.http` y el nuevo `global.json`. La solución se crea en el paso 2 y la carpeta `docs` en el paso 3.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.
- Copiar desde el punto 2.3 en lugar del 2.4: se pierde el contenido del último punto del módulo 2.

PASO 3.1.2 — Abrir la solución copiada en Visual Studio 2026

Abrir la solución copiada permite trabajar en el punto 3.1 sobre la base del punto 2.4. La compilación verifica que la copia es correcta.

1. Abre en Visual Studio la solución `AceriaApp.sln` de `3.1-capas-aplicacion`; si la primera etapa aún no la tiene, crea una solución con `dotnet new sln -n AceriaApp`.
2. Desde la raíz del punto ejecuta `dotnet sln AceriaApp.sln add src/AceriaApp.Api/AceriaApp.Api.csproj tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj` para la etapa inicial; en las siguientes etapas la solución viene heredada.
3. Verifica con `dotnet sln AceriaApp.sln list` que todas las rutas corresponden a proyectos presentes en la carpeta de esta etapa.
4. Ejecuta `dotnet build AceriaApp.sln -c Release` antes de tocar las clases y guarda su resultado como punto de comparación.

Verificación concreta. La solución abierta contiene los proyectos existentes de la etapa anterior y compila antes de los nuevos archivos.

Diagnóstico si falla.

- Abrir la solución del punto 2.4 en lugar de la del punto 3.1: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.

- Tener la API en ejecución mientras Windows bloquea sus archivos de salida: detén la aplicación antes de recompilar.

PASO 3.1.3 — Preparar la carpeta `docs` para la documentación de arquitectura

El punto 3.1 incorpora documentación específica de arquitectura a la API de M02. Crear `docs` en la copia mantiene intacto el punto anterior.

1. Abre `docs` dentro de `3.1-capas-aplicacion`. Si es la primera etapa y no existe, créala: botón derecho sobre la raíz del punto y `Agregar > Nueva carpeta`.
2. La base M02 2.4 conserva su documentación en `README.md`; no incorpora una carpeta `docs`. Comprueba que ese README sigue presente en la copia.
3. Anota los dos documentos nuevos que crearás durante este punto: `capas-aplicacion.md` y `ADR-010-capas-aplicacion.md`.
4. Deja `docs` preparada y vacía. El documento de diseño se añadirá en el paso 4 y el ADR en el paso 20; las etapas posteriores heredarán ambos.

Verificación concreta. la carpeta `docs` existe dentro de `3.1-capas-aplicacion`, está vacía al terminar este paso y los archivos de M02 permanecen intactos.

Diagnóstico si falla.

- Crear `docs` dentro de M02: comprueba que trabajas en la copia del punto 3.1.
- Eliminar `README.md`: conserva ese archivo hasta su actualización guiada en el paso 19.
- Repetir la creación si `docs` ya existe: abre la carpeta y revisa su contenido antes de continuar.

PASO 3.1.4 — Crear el documento de capas de la aplicación en `docs`

El documento de capas de la aplicación es el artefacto de `docs` específico del punto 3.1. Se crea en este punto y conserva intactos los archivos heredados de M02 2.4.

1. En la carpeta `docs` de `3.1-capas-aplicacion` crea `capas-aplicacion.md` mediante `Agregar > Nuevo elemento` o el editor Markdown.
2. Copia el documento de este paso y estudia sus apartados. Relaciónalo con los bloques de teoría del punto 3.1; no debe limitarse a una lista de nombres.
3. Distingue los componentes del catálogo arquitectónico de las clases y los servicios implementados en el proyecto.
4. Guarda el Markdown, abre su vista previa y anota dos decisiones observables que podrás verificar posteriormente con los tests.

Archivo que debe quedar tras este paso: `docs/capas-aplicacion.md`

```
# Capas de la aplicación — Punto 3.1

## Capas

1. Presentación.
2. Aplicación.
3. Dominio.
4. Infraestructura.

## Capa de dominio

Contiene entidades, value objects, agregados y reglas de negocio. No depende de ninguna otra capa.

## Capa de aplicación

Contiene casos de uso y orquestación. Depende del dominio. Define interfaces que la infraestructura
implementa.

## Capa de infraestructura
```

```
Contiene implementaciones concretas: repositorios, clientes HTTP, publicadores de eventos. Depende de las
interfaces definidas en las capas interiores.

## Capa de presentación

Contiene endpoints y controladores. Traduce entre el protocolo y los casos de uso. Depende de la aplicación.

## Regla de dependencia

Las dependencias se dirigen hacia el interior:

- Presentación → Aplicación → Dominio.
- Infraestructura → Aplicación → Dominio.
- Dominio no depende de nadie.

## Inversión de dependencias

Las interfaces se definen en las capas interiores y se implementan en las exteriores. El contenedor de
dependencias inyecta la implementación.

## Arquitectura hexagonal

Núcleo central con dominio y aplicación, rodeado de puertos y adaptadores. Un mismo puerto puede tener varios
adaptadores.

## Aplicación al proyecto del curso

| Capa | Contenido |
|------|-----------|
| Dominio | Entidades y reglas de negocio |
| Aplicación | Casos de uso e interfaces de repositorio |
| Infraestructura | Implementaciones de repositorio y EF Core |
| Presentación | Endpoints de Minimal API |

## Corrección de compatibilidad con el repositorio vigente

La implementación de partida no es `AceriaApp.Monolito.Api`, sino la Minimal API del proyecto
`M02_APIS_REST_ASPNET_CORE/PROYECTO/2.4-documentacion-openapi`. Los ejemplos de esta ficha describen
conceptos de diseño; su implementación real se incorpora al código en 3.3 y 3.4, con puertos de aplicación e
infraestructura SQL Server. No se considera registrado un adaptador solo por mencionarlo en un catálogo.
```

Verificación concreta. el archivo `docs\capas-aplicacion.md` existe con las nueve secciones.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 3.1.5 — Crear la carpeta `Capas` en el proyecto `AceriaApp.Api`

La carpeta `Capas` albergará las clases de validación de la regla de dependencia. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el Explorador de soluciones, dentro del proyecto `AceriaApp.Api`, crea la carpeta `Capas` si no existe.
2. Comprueba que el directorio físico es `src/AceriaApp.Api/Capas` y no está situado dentro del proyecto de tests.
3. Prepara los tres archivos de este punto: `Capa.cs`, `ValidadorReglaDependencia.cs` y `CatalogoCapas.cs`, que añadirás en los siguientes pasos.
4. No confundas estos modelos de catálogo arquitectónico con la implementación real del puerto de órdenes, que aparecerá más adelante.

Verificación concreta. la carpeta `Capas` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 3.1.6 — Crear la clase `Capa`

La clase `Capa` describe dependencias permitidas dentro del ejercicio. No crea una biblioteca ni demuestra que el código compilado respete esa restricción: ofrece datos al validador.

1. Crea `src/AceriaApp.Api/Capas/Capa.cs` y escribe el código íntegro mostrado en el bloque asociado al paso.
2. Comprueba que el namespace es `AceriaApp.Api.Capas`, correspondiente a la carpeta de este punto.
3. Lee cada propiedad y diferencia valores de configuración de objetos de dominio. No añadas dependencias SQL o ASP.NET Core a este modelo de catálogo.
4. Guarda y compila con `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`; si faltan tipos, revisa el namespace y los usings implícitos.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/Capas/Capa.cs`

```
namespace AceriaApp.Api.Capas;

public class Capa
{
    public string Nombre { get; init; } = string.Empty;
    public IReadOnlyList<string> DependenciasPermitidas { get; init; } = Array.Empty<string>();
}
```

Verificación concreta. la clase `Capa` compila sin errores y expone dos propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar `Nombre` con `string.Empty`: el compilador emite advertencia de nulabilidad.
- No inicializar `DependenciasPermitidas` con `Array.Empty<string>()`: el compilador emite advertencia.

PASO 3.1.7 — Crear la clase `ValidadorReglaDependencia`

`ValidadorReglaDependencia` debe explicar por qué una relación entre capas se acepta o rechaza. Sus pruebas acreditan el comportamiento de la regla declarada, no la inspección automática de todos los proyectos.

1. Abre o crea `src/AceriaApp.Api/Capas/ValidadorReglaDependencia.cs` según el bloque de C# de este paso.
2. Escribe la clase pública `ValidadorReglaDependencia` y el record de resultado; comprueba sus tipos de entrada y las ramas de error.
3. Recorre con el depurador un caso válido y otro inválido. Localiza dónde se agrega cada mensaje y qué condición determina el resultado final.
4. Guarda, compila y evita añadir el mismo validador en otro fichero. Las pruebas de esta regla se crearán en el paso 14.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/Capas/ValidadorReglaDependencia.cs`

```
namespace AceriaApp.Api.Capas;

public class ValidadorReglaDependencia
```

```
{
    private readonly Dictionary<string, IReadOnlyList<string>> _reglas;

    public ValidadorReglaDependencia(Dictionary<string, IReadOnlyList<string>> reglas)
    {
        _reglas = reglas;
    }

    public ResultadoDependencia Validar(string capaOrigen, string capaDestino)
    {
        var errores = new List<string>();

        if (!_reglas.ContainsKey(capaOrigen))
        {
            errores.Add($"Capa origen desconocida: '{capaOrigen}'.");
            return new ResultadoDependencia(false, errores);
        }

        if (!_reglas[capaOrigen].Contains(capaDestino))
        {
            errores.Add($"La capa '{capaOrigen}' no puede depender de '{capaDestino}'.");
        }

        return new ResultadoDependencia(errores.Count == 0, errores);
    }
}

public record ResultadoDependencia(bool EsValida, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorReglaDependencia` compila sin errores y expone un constructor, un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `Dictionary` y `List<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` no se resuelve.
- No comprobar si la capa origen existe: lanza `KeyNotFoundException`.

PASO 3.1.8 — Añadir los `using` necesarios en `ValidadorReglaDependencia`

Los tipos `Dictionary` y `List<string>` pertenecen a `System.Collections.Generic`. El método `Contains` pertenece a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. Abre `src/AceriaApp.Api/Capas/ValidadorReglaDependencia.cs` y localiza las directivas `using` de la parte superior.
2. Verifica que `System.Collections.Generic`, `System.Linq` y `System` están disponibles según las colecciones o métodos usados; con ImplicitUsings activado algunos llegan automáticamente.
3. Añade solo el import que realmente falte. No copies una segunda clase ni sustituyas la implementación del validador por un comentario.
4. Guarda y compila. Deja registrado qué miembro concreto requería el namespace, como `List<string>`, `Where` o `Contains`.

Verificación concreta. `ValidadorReglaDependencia.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `Contains` no se resuelve.
- Añadir los usings después del namespace: el compilador no los reconoce.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 3.1.9 — Crear la clase `CatalogoCapas`

`CatalogoCapas` reúne reglas conocidas para clasificarlas de forma consistente. Una lista bien formada no detecta por sí sola una referencia real a ASP.NET Core o EF Core dentro del dominio.

1. Crea `src/AceriaApp.Api/Capas/CatalogoCapas.cs` con el catálogo y los valores de ejemplo indicados en el código del laboratorio.
2. Identifica las reglas o registros que el catálogo declara y cuáles pertenecen únicamente al ejercicio conceptual; no los des por registrados en DI.
3. Comprueba que el catálogo utiliza los nombres de capas, puertos o proyectos acordados y que no declara interfaces imposibles de resolver en tiempo de compilación.
4. Guarda y compila la API. Relaciona cada ejemplo con una prueba del paso 15.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/Capas/CatalogoCapas.cs`

```
namespace AceriaApp.Api.Capas;

public class CatalogoCapas
{
    public static Dictionary<string, IReadOnlyList<string>> ReglasPorDefecto()
    {
        return new Dictionary<string, IReadOnlyList<string>>
        {
            { "Presentacion", new[] { "Aplicacion" } },
            { "Aplicacion", new[] { "Dominio" } },
            { "Infraestructura", new[] { "Aplicacion", "Dominio" } },
            { "Dominio", Array.Empty<string>() }
        };
    }
}
```

Verificación concreta. la clase `CatalogoCapas` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `Dictionary` no se resuelve.
- Olvidar el `using System;`: `Array.Empty` no se resuelve.
- Devolver `Dictionary` en lugar de `IReadOnlyDictionary`: expone la implementación interna.

PASO 3.1.10 — Añadir los `using` necesarios en `CatalogoCapas`

El tipo `Dictionary` pertenece a `System.Collections.Generic` y `Array.Empty` pertenece a `System`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. Abre `src/AceriaApp.Api/Capas/CatalogoCapas.cs` y observa las colecciones e importaciones utilizadas por el catálogo.
2. Revisa `using System;`, `using System.Linq;` y `using System.Collections.Generic;` cuando corresponda; conserva solo lo que la compilación necesite.
3. No añadas dependencias tecnológicas nuevas al dominio para resolver tipos básicos ni vuelvas a crear el archivo de catálogo.
4. Compila y confirma que los métodos del catálogo pueden llamarse desde el proyecto de tests sin errores de namespace.

Verificación concreta. `CatalogoCapas.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System`: `Array.Empty` no se resuelve.
- Añadir los usings después del namespace: el compilador no los reconoce.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 3.1.11 — Crear el proyecto de tests del punto 3.1

El punto 3.1 añade un proyecto de tests específico para las clases de capas. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Dentro de `tests` crea el proyecto xUnit `AceriaApp.Capas.Tests` con destino `net8.0`. No modifiques ni sustituyas `AceriaApp.Api.Tests`, que contiene las pruebas HTTP heredadas de M02.
2. Crea el archivo `tests/AceriaApp.Capas.Tests/AceriaApp.Capas.Tests.csproj` y escribe el XML completo del bloque del laboratorio, con xUnit y FluentAssertions 6.12.2.
3. Comprueba que incluye un `ProjectReference` hacia `../../src/AceriaApp.Api/AceriaApp.Api.csproj`; agrega el nuevo proyecto con `dotnet sln AceriaApp.sln add tests/AceriaApp.Capas.Tests/AceriaApp.Capas.Tests.csproj`.
4. Ejecuta `dotnet restore AceriaApp.sln` y comprueba que no se ha generado un test `UnitTest1` no previsto.

Archivo que debe quedar tras este paso: `tests/AceriaApp.Capas.Tests/AceriaApp.Capas.Tests.csproj`

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsTestProject>true</IsTestProject><IsPackable>false</IsPackable></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El proyecto de tests `AceriaApp.Capas.Tests` restaura, referencia a `AceriaApp.Api` y no sustituye ninguna suite heredada.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 2.4: viola la regla de no modificar archivos del punto anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 3.1.12 — Añadir la referencia del proyecto de tests a la API `AceriaApp.Api`

La referencia permite que el proyecto de tests acceda a las clases de `Capas` del proyecto `AceriaApp.Api`.

1. En `tests/AceriaApp.Capas.Tests/AceriaApp.Capas.Tests.csproj` localiza la referencia al proyecto `AceriaApp.Api`.
2. Verifica la dirección de la dependencia: los tests referencian a la API, nunca la API al proyecto de tests.
3. Ejecuta `dotnet sln AceriaApp.sln list` y verifica que el nuevo proyecto aparece junto a los heredados, con la ruta física correcta.
4. No dupliques `ProjectReference` si la referencia ya está preparada en el XML del paso 11.

Verificación concreta. El proyecto de tests `AceriaApp.Capas.Tests` restaura, referencia a `AceriaApp.Api` y no sustituye ninguna suite heredada.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.

- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 3.1.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Abre el `.csproj` de `AceriaApp.Capas.Tests` y localiza el `PackageReference` a `FluentAssertions` versión 6.12.2.
2. Ejecuta `dotnet list tests/AceriaApp.Capas.Tests/AceriaApp.Capas.Tests.csproj package` y comprueba que el paquete consta en el proyecto correcto.
3. Relaciona el método `Should()` de los ejemplos de tests con FluentAssertions, y `[Fact]` con xUnit.
4. No instales una segunda versión del mismo paquete: basta con restaurar NuGet desde la solución.

Verificación concreta. El proyecto de tests `AceriaApp.Capas.Tests` restaura, referencia a `AceriaApp.Api` y no sustituye ninguna suite heredada.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 3.1.14 — Crear el archivo `ValidadorReglaDependenciaTests.cs`

Los tests verifican que el validador acepta las dependencias que cumplen la regla y rechaza las que la violan.

1. Crea `tests/AceriaApp.Capas.Tests/ValidadorReglaDependenciaTests.cs` y escribe completo el primer bloque de pruebas de este punto.
2. Revisa el `using AceriaApp.Api.Capas` para que coincida con el namespace actualizado y conserva `using Xunit;` y `using FluentAssertions;`.
3. Localiza los casos correctos e incorrectos del validador y comprueba que las aserciones describen resultados de código ejecutable.
4. Guarda, compila y ejecuta el test específico. No cambies las expectativas para ocultar una regla equivocada.

Archivo que debe quedar tras este paso: `tests/AceriaApp.Capas.Tests/ValidadorReglaDependenciaTests.cs`

```
using AceriaApp.Api.Capas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Capas.Tests;

public class ValidadorReglaDependenciaTests
{
    private readonly ValidadorReglaDependencia _validador = new(CatalogoCapas.ReglasPorDefecto());

    [Fact]
    public void Validar_PresentacionDependeDeAplicacion_DevuelveValida()
    {
        var resultado = _validador.Validar("Presentacion", "Aplicacion");

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_AplicacionDependeDeDominio_DevuelveValida()
    {
```

```
        var resultado = _validador.Validar("Aplicacion", "Dominio");

        resultado.EsValida.Should().BeTrue();
    }

    [Fact]
    public void Validar_DominioDependeDeAplicacion_DevuelveError()
    {
        var resultado = _validador.Validar("Dominio", "Aplicacion");

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no puede depender"));
    }

    [Fact]
    public void Validar_CapaOrigenDesconocida_DevuelveError()
    {
        var resultado = _validador.Validar("CapaInventada", "Dominio");

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("desconocida"));
    }
}
```

Verificación concreta. Los tests de `AceriaApp.Capas.Tests` compilan y ejecutan las aserciones propuestas para `Capas`.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Capas;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de capa origen desconocida: el mensaje de error no se verifica.

PASO 3.1.15 — Crear el archivo `CatalogoCapasTests.cs`

Los tests verifican que el catálogo de capas contiene las cuatro capas esperadas y que las reglas de dependencia son las correctas.

1. Crea `tests/AceriaApp.Capas.Tests/CatalogoCapasTests.cs` y escribe el segundo bloque de pruebas del catálogo.
2. Comprueba que las llamadas a los métodos de ejemplo de `{catalog}` son compatibles con las propiedades que realmente expone el código.
3. Guarda y ejecuta `dotnet test tests/AceriaApp.Capas.Tests/AceriaApp.Capas.Tests.csproj -c Release`, de forma que se ejecuten todas las pruebas nuevas.
4. Verifica que las aserciones prueban los valores declarados, pero no confunde un catálogo didáctico con un registro en tiempo de ejecución.

Archivo que debe quedar tras este paso: `tests/AceriaApp.Capas.Tests/CatalogoCapasTests.cs`

```
using AceriaApp.Api.Capas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Capas.Tests;

public class CatalogoCapasTests
{
    [Fact]
    public void ReglasPorDefecto_ContieneCuatroCapas()
    {
        var reglas = CatalogoCapas.ReglasPorDefecto();

        reglas.Should().HaveCount(4);
        reglas.Should().ContainKey("Presentacion");
        reglas.Should().ContainKey("Aplicacion");
        reglas.Should().ContainKey("Infraestructura");
        reglas.Should().ContainKey("Dominio");
```

```
    }

    [Fact]
    public void ReglasPorDefecto_DominioNoDependeDeNadie()
    {
        var reglas = CatalogoCapas.ReglasPorDefecto();

        reglas["Dominio"].Should().BeEmpty();
    }

    [Fact]
    public void ReglasPorDefecto_InfraestructuraDependeDeAplicacionYDominio()
    {
        var reglas = CatalogoCapas.ReglasPorDefecto();

        reglas["Infraestructura"].Should().Contain("Aplicacion");
        reglas["Infraestructura"].Should().Contain("Dominio");
    }
}
```

Verificación concreta. Los tests de `AceriaApp.Capas.Tests` compilan y ejecutan las aserciones propuestas para `Capas`.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Capas;`: las clases no se resuelven.
- Usar `Assert.Equal` en lugar de `Should().Be()`: el curso usa FluentAssertions.
- No verificar la regla del dominio: la capa más interna no se comprueba.

PASO 3.1.16 — Compilar la solución completa

Compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

1. Abre la terminal junto a `AceriaApp.sln` y ejecuta `dotnet build AceriaApp.sln -c Release` después de introducir todos los archivos del punto.
2. Comprueba que compilan la API de M02, el proyecto nuevo de tests y todos los proyectos de etapas anteriores.
3. Si aparece un error, identifica el primer archivo y la primera línea antes de tocar otra clase. Revisa referencias y namespaces.
4. Registra la salida de la compilación y no avances hasta obtener cero errores.

Verificación concreta. La solución completa compila en Release con cero errores. La capa de dominio no referencia EF Core ni ASP.NET Core.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 3.1.17 — Ejecutar todos los tests del punto 3.1

Ejecutar los tests del punto 3.1 verifica que las clases de capas funcionan correctamente.

1. Ejecuta `dotnet test tests/AceriaApp.Capas.Tests/AceriaApp.Capas.Tests.csproj -c Release` desde la raíz del punto.
2. Comprueba que los métodos `[Fact]` de `ValidadorReglaDependenciaTests` y `CatalogoCapasTests` pasan; el resumen numérico exacto aparece en la tabla de resultados de este módulo.

3. Ejecuta un caso inválido del validador en el depurador y relaciona el mensaje de error con la línea que lo produce.
4. Conserva la salida de tests como evidencia y confirma que los resultados no dependen de SQL Server para evaluar reglas puras.

Verificación concreta. La suite nueva de 3.1 supera 7 pruebas de catálogo; en 3.4 se añaden además 4 pruebas HTTP del adaptador real.

Diagnóstico si falla.

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 3.1.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 3.1 pasan conjuntamente.

1. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después de compilar toda la solución en Release.
2. Comprueba que pasan los tests de M02 y todos los nuevos del módulo M03 acumulados hasta este punto.
3. Si falla alguna suite previa, vuelve a comprobar si el último cambio alteró una dependencia o una configuración del servidor.
4. Registra las pruebas totales, fallidas y omitidas y compáralas con el resultado previsto para esta etapa.

Verificación concreta. Pruebas acumuladas previstas para 3.1: 23, 0 fallidas. Comprueba estos valores en la salida de `dotnet test`.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

PASO 3.1.19 — Crear el README del punto 3.1

El README del punto 3.1 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 2.4.

1. Abre `README.md` en la raíz del punto `3.1-capas-aplicacion` y sustitúyelo por el README específico del bloque de este paso.
2. Comprueba que los nombres de proyectos y las pruebas acumuladas coinciden con la solución y con la salida de `dotnet test` de esta etapa.
3. Revisa la ubicación del documento `docs` y los comandos `dotnet build` y `dotnet test` que utilizará el alumno.
4. Guarda la versión del punto actual sin sobrescribir el README de la etapa anterior.

Archivo que debe quedar tras este paso: `README.md`

```
# Punto 3.1 — Estructura del módulo 3 y del punto 3.1

**Curso:** Microservicios y arquitectura hexagonal con ASP.NET Core 8
**Código de referencia:** AceriaApp, en continuidad con M02 2.4.
```

```
## Objetivo

el documento de capas de la aplicación es el artefacto de `docs` específico del punto 3.1. Se crea en este
punto y conserva intactos los archivos heredados de M02 2.4.

## Estado verificado del punto

- 3 proyectos en la solución `AceriaApp.sln` al finalizar este punto.
- 23 pruebas xUnit acumuladas previstas (incluyendo las 16 de M02 y las nuevas del módulo 3).
- Clases del catálogo `Capas` en `src/AceriaApp.Api/Capas`.
- Documentación `docs/capas-aplicacion.md` y decisión `docs/ADR-010-capas-aplicacion.md`.
- Los endpoints REST del módulo 2 se conservan sin cambios funcionales.

## Cómo ejecutarlo

1. Abrir el archivo `AceriaApp.sln` de esta carpeta en Visual Studio, con SDK .NET 8.
2. Ejecutar `dotnet restore AceriaApp.sln` y después `dotnet build AceriaApp.sln -c Release`.
3. Ejecutar `dotnet test AceriaApp.sln -c Release --no-build`. La suite M03 se construye por adición desde
M02.
4. La aplicación principal utiliza SQL Server Express LocalDB; los tests HTTP mantienen SQL Server y crean
bases exclusivas independientes mediante `BaseSqlServer.Crear()`.
5. En 3.4, los endpoints hexagonales reales están en `/api/v1/hexagonal/ordenes` y utilizan un repositorio EF
Core a través de un puerto de aplicación.

## Qué se ha añadido

la clase `Capa` modela una capa con su nombre y las capas de las que puede depender. Es la unidad de
información del validador de la regla de dependencia.
```

Verificación concreta. `README.md` describe correctamente los 3 proyectos y 23 pruebas acumuladas del curso vigente.

Diagnóstico si falla.

- Modificar el README del punto 2.4: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 2.4: rompe la separación.

PASO 3.1.20 — Crear el ADR-010 del punto 3.1

El ADR-010 documenta la decisión de adoptar la organización por capas y la regla de dependencia. Es un artefacto nuevo del punto 3.1 que no modifica los ADR existentes.

1. En la carpeta `docs` crea `ADR-010-capas-aplicacion.md` y copia el documento de decisión del bloque del laboratorio.
2. Lee Estado, Contexto, Decisión, Alternativas y Consecuencias y relaciona el ADR con el código nuevo de la etapa.
3. Explica por qué no basta con declarar nombres de capas si el código puede saltarse la interfaz o registrar una dependencia incorrecta.
4. Comprueba que los ADR anteriores están presentes y no han sido sobrescritos.

Archivo que debe quedar tras este paso: `docs/ADR-010-capas-aplicacion.md`

```
# ADR-010: Organización por capas y regla de dependencia

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha cerrado el módulo 2 con una API REST documentada. Es necesario reorganizar el código
en capas para preparar la evolución hacia arquitectura hexagonal.

## Decisión
Se adopta la organización por capas: presentación, aplicación, dominio e infraestructura. La regla de
dependencia se aplica de forma estricta: las dependencias se dirigen hacia el interior. La inversión de
dependencias permite que la aplicación dependa de interfaces y que la infraestructura las implemente. La
```

```
validación de la regla se realiza con un validador automático.

## Alternativas consideradas
- Mantener la organización plana: descartado por dificultad de evolución.
- Aplicar solo la separación de dominio: descartado por incompleto.
- Usar capas apiladas sin inversión de dependencias: descartado por acoplamiento.

## Consecuencias
- Positivas: separación de responsabilidades, testabilidad, mantenibilidad.
- Negativas: la organización requiere disciplina en la aplicación de la regla.
- Mitigación: validación automática de la regla con el validador.

## Aplicación en AceriaApp

Se parte de la API real y probada de M02, se conservan sus endpoints y se añaden reglas de dependencia
verificadas en código. Los proyectos de Dominio, Aplicación e Infraestructura deben obedecer la dirección de
referencias; en 3.4 se expone un nuevo adaptador HTTP funcional. El catálogo describe las relaciones
arquitectónicas; cada implementación se comprueba en su proyecto correspondiente.
```

Verificación concreta. el archivo `docs\ADR-010-capas-aplicacion.md` existe con las cinco secciones.

Diagnóstico si falla.

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-010`: rompe la convención.

PASO 3.1.21 — Verificar que no se ha modificado ningún archivo del punto 2.4

La comparación con `fc` verifica que todos los documentos del punto 2.4 no se han modificado.

1. Localiza el mismo documento heredado tanto en `M02_APIS_REST_ASPNET_CORE/PROYECTO/2.4-documentacion-openapi` como en `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.1-capas-aplicacion`.
2. Compara sus hashes SHA-256 o usa un diff y confirma que no se reescribió al introducir el nuevo punto.
3. Distingue de él los archivos añadidos en este punto: catálogo, validador, pruebas y documentación nueva.
4. Documenta cualquier diferencia inesperada; el nuevo código no exige alterar los archivos de etapas anteriores.

Verificación concreta. Los archivos heredados de `docs` y los endpoints M02 no han sido sobrescritos.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Olvidar el flag `/L`: `fc` compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 3.1.22 — Verificar la estructura del punto 3.1

La verificación estructural confirma que el punto 3.1 añade archivos nuevos sin modificar los existentes.

1. Desde la raíz de `M03_ARQUITECTURA_HEXAGONAL/PROYECTO` ejecuta `tree /F 3.1-capas-aplicacion` en Windows.
2. Localiza `src/AceriaApp.Api/Capas`, `tests/AceriaApp.Capas.Tests`, `README.md` y `docs/capas-aplicacion.md`.
3. Comprueba los proyectos adicionales de dominio, aplicación e infraestructura si ya fueron introducidos por el punto 3.3.
4. Distingue los archivos del proyecto de los temporales `obj/` y `bin/`; ambos deben quedar excluidos de la entrega.

Verificación concreta. Los archivos que aparecen en el árbol corresponden a los fragmentos creados hasta este paso, y no hay código pendiente en carpetas fuera de la solución.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 2.4: viola la regla de no modificar.
- Omitir la carpeta `Capas`: los tests no encuentran las clases.

PASO 3.1.23 — Verificar el `.gitignore` del punto 3.1

El `.gitignore` se hereda del punto 2.4 sin modificaciones. La verificación confirma que no se ha tocado.

1. Abre el `.gitignore` de la etapa; si falta, crea uno que excluya `.vs/`, `bin/`, `obj/`, archivos personales y resultados de prueba.
2. Ejecuta `git status --short` en tu copia de trabajo y comprueba que no aparecen archivos binarios ni credenciales.
3. No ignores `src`, `tests`, `docs` ni las soluciones `.sln`, pues son parte del entregable reproducible.
4. Conserva el `.gitignore` heredado sin borrarlo ni modificar el de etapas anteriores.

Archivo que debe quedar tras este paso: `.gitignore`

```
**/bin/
**/obj/
.vs/
*.user
TestResults/
*.trx
```

Verificación concreta. No se incluyen `bin`, `obj`, contraseñas ni resultados de pruebas en el estado Git.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 2.4: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 3.1.24 — Hacer el commit del punto 3.1 y verificar el estado final

El commit del punto 3.1 registra el estado acumulado del proyecto. Los commits de cada punto permiten comparar el estado del proyecto entre puntos consecutivos y entre módulos completos.

1. Antes de terminar 3.1, vuelve a ejecutar `dotnet build AceriaApp.sln -c Release` y `dotnet test AceriaApp.sln -c Release --no-build`.
2. Lee el reto de ampliación incluido tras los pasos obligatorios; si lo realizas, utiliza una copia aparte y deja los tests base intactos.
3. Si tu copia está bajo Git, ejecuta `git status`, añade solo lo trabajado y crea un commit descriptivo, sin modificar la rama canónica del profesor.
4. Conserva el resumen de pruebas y compara el proyecto final con la carpeta de referencia del punto; ambas deben representar el mismo código.

Verificación concreta. El punto termina con compilación y suite de pruebas completa, sin modificar la referencia de la etapa anterior.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.

- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Esta ampliación añade una operación de resumen al validador y una prueba de su resultado. **Realízala después de superar la etapa básica y sus tests.** Incorpora cada fragmento en la clase indicada y conserva el namespace del proyecto en el que trabajas.

Reto: añadir un método `Resumen` a `ValidadorReglaDependencia` que devuelva el número de capas y de dependencias permitidas del catálogo, y un test que verifique el cálculo.

Paso 1: En `ValidadorReglaDependencia.cs`, añadir el siguiente `record` al final del archivo:

```
public record ResumenDependencias(int Capas, int DependenciasPermitidas);
```

Paso 2: Explicar línea por línea:

- **Línea 1:** `public record ResumenDependencias(` → declara un contrato inmutable.
- **Línea 2:** `int Capas,` → número de capas del catálogo.
- **Línea 3:** `int DependenciasPermitidas);` → número total de dependencias permitidas.

Paso 3: En `ValidadorReglaDependencia.cs`, añadir el siguiente método dentro de la clase:

```
public ResumenDependencias Resumen()
{
    var capas = _reglas.Count;
    var dependencias = _reglas.Sum(r => r.Value.Count);
    return new ResumenDependencias(capas, dependencias);
}
```

Paso 4: Explicar línea por línea:

- **Línea 1:** `public ResumenDependencias Resumen()` → declara el método.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `var capas = _reglas.Count;` → cuenta las capas del catálogo.
- **Línea 4:** `var dependencias = _reglas.Sum(r => r.Value.Count);` → suma las dependencias permitidas.
- **Línea 5:** `return new ResumenDependencias(capas, dependencias);` → devuelve el resumen.
- **Línea 6:** `}` → cierra el cuerpo.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorReglaDependenciaTests.cs`, añadir el siguiente test:

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CuentaCorrectamente()
{
    var resumen = _validador.Resumen();

    resumen.Capas.Should().Be(4);
    resumen.DependenciasPermitidas.Should().Be(4);
}
```

Paso 7: Explicar línea por línea:

- **Línea 1:** `[Fact]` → marca el método como test de xUnit.
- **Línea 2:** `public void Resumen_ConCatalogoPorDefecto_CuentaCorrectamente()` → declara el test.
- **Línea 3:** `{` → abre el cuerpo.

- **Línea 4:** `var resumen = _validador.Resumen();` → invoca el método.
- **Línea 5:** `resumen.Capas.Should().Be(4);` → verifica el número de capas.
- **Línea 6:** `resumen.DependenciasPermitidas.Should().Be(4);` → verifica el número de dependencias.
- **Línea 7:** `}` → cierra el cuerpo.

Paso 8: Ejecutar `dotnet test tests\AceriaApp.Capas.Tests\AceriaApp.Capas.Tests.csproj` y verificar que pasan los ocho tests.

Paso 9: Verificar que el método `Resumen` no modifica ningún archivo del punto 2.4.

Paso 10: Hacer commit con `git commit -m "Reto 3.1: método Resumen en ValidadorReglaDependencia"`.

Verificación: la salida de `dotnet test` muestra `Passed: 8`.

Resultado verificable y cierre del punto 3.1

Código final de referencia: `PROYECTO/3.1-capas-aplicacion/`. El contenido que debe crear el alumno está especificado por los bloques con su ruta exacta y el tipo de modificación. Comprueba el resultado compilando y ejecutando las pruebas de la etapa.

Continuidad: las dependencias existentes y la documentación heredada se conservan; el avance del punto 3.1 prepara el siguiente sin reescribir la etapa anterior.

Punto 3.2 — Puertos y adaptadores

Razonar sobre puertos y adaptadores sin confundir su descripción con la implementación

Un puerto expresa una operación que la aplicación necesita, mientras que un adaptador resuelve esa colaboración con una tecnología concreta. En 3.2 estudiaremos esa relación primero con clases y reglas de análisis, sin afirmar que un catálogo de nombres haya conectado ya el dominio con la infraestructura.

Al avanzar desde 3.1, mantendremos los endpoints y el SQL Server de M02. Los nuevos tests mostrarán qué asociaciones acepta el validador y cuáles rechaza. Cuando necesitemos demostrar la inversión de dependencias en ejecución, verificaremos también las referencias de proyectos y el flujo real de un caso de uso.

Objetivos del punto. el documento de puertos y adaptadores es el artefacto de `docs` específico del punto 3.2. No modifica los catorce documentos existentes del punto 3.1.

Carpeta de trabajo: `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.2-puertos-adaptadores`. Esta etapa continúa la anterior sin alterar su copia. El código de la referencia es un resultado final, no una solución que el alumno deba pegar al inicio.

Criterio de aceptación: seguir los 24 pasos, compilar y ejecutar los tests; comprobar los resultados indicados y registrar las actividades manuales. Al finalizar, la solución debe reproducir el comportamiento descrito.

PASO 3.2.1 — Crear la carpeta del punto 3.2 a partir del punto 3.1

Cada punto del curso parte del contenido del punto anterior. El punto 3.2 se construye sobre el punto 3.1 copiando su contenido y añadiendo archivos nuevos sin modificar los existentes.

1. En el Explorador de archivos localiza `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.1-capas-aplicacion`. Comprueba que es el estado anterior completo, con los endpoints y tests ya construidos.
2. Crea una carpeta personal `3.2-puertos-adaptadores` bajo `M03_ARQUITECTURA_HEXAGONAL/PROYECTO`. Copia en ella el contenido del punto anterior, sin cambiarlo en su origen.
3. Desde la raíz de `3.2-puertos-adaptadores` comprueba que tienes `src/AceriaApp.Api`, `tests/AceriaApp.Api.Tests`, `AceriaApp.http` y los archivos de configuración.
4. No copies `bin/`, `obj/` ni credenciales. Para comparar puedes usar los proyectos de referencia; los nuevos archivos de este punto se introducirán solo cuando llegues a su paso.

Verificación concreta. al ejecutar `dir 3.2-puertos-adaptadores`, deben aparecer `AceriaApp.sln`, `README.md`, `src`, `tests` y `docs`.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

PASO 3.2.2 — Abrir la solución copiada en Visual Studio 2026

Abrir la solución copiada permite trabajar en el punto 3.2 sobre la base del punto 3.1. La compilación verifica que la copia es correcta.

1. Abre en Visual Studio la solución `AceriaApp.sln` de `3.2-puertos-adaptadores`; si la primera etapa aún no la tiene, crea una solución con `dotnet new sln -n AceriaApp`.
2. Desde la raíz del punto ejecuta `dotnet sln AceriaApp.sln add src/AceriaApp.Api/AceriaApp.Api.csproj tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj` para la etapa inicial; en las siguientes etapas la solución viene heredada.
3. Verifica con `dotnet sln AceriaApp.sln list` que todas las rutas corresponden a proyectos presentes en la carpeta de esta etapa.
4. Ejecuta `dotnet build AceriaApp.sln -c Release` antes de tocar las clases y guarda su resultado como punto de comparación.

Verificación concreta. La solución abierta contiene los proyectos existentes de la etapa anterior y compila antes de los nuevos archivos.

Diagnóstico si falla.

- Abrir la solución del punto 3.1 en lugar de la del punto 3.2: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 3.1 en ejecución: el puerto queda ocupado y la compilación falla.

PASO 3.2.3 — Verificar que la carpeta `docs` contiene los documentos del punto 3.1

El punto 3.2 parte del contenido del punto 3.1, que ya incluye la carpeta `docs` con catorce documentos. No se modifica ningún documento existente.

1. Abre `docs` dentro de `3.2-puertos-adaptadores`. Si es la primera etapa y no existe, créala: botón derecho sobre la raíz del punto y `Agregar > Nueva carpeta`.
2. Comprueba que los documentos de la etapa anterior no se han modificado. A partir de aquí se añade una nueva decisión y un nuevo documento de diseño.
3. Anota los nombres de los documentos que recibes y los que crearás: `puertos-adaptadores.md` y `ADR-011-puertos-adaptadores.md`.
4. No reescribas un ADR anterior para explicar un diseño nuevo. Comprueba que los documentos heredados mantienen su contenido entre etapas.

Verificación concreta. la carpeta `docs` contiene los catorce documentos del punto 3.1 sin modificaciones.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 3.1.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 3.2.4 — Crear el documento de puertos y adaptadores en `docs`

El documento de puertos y adaptadores es el artefacto de `docs` específico del punto 3.2. No modifica los catorce documentos existentes del punto 3.1.

1. En la carpeta `docs` de `3.2-puertos-adaptadores` crea `puertos-adaptadores.md` mediante `Agregar > Nuevo elemento` o el editor Markdown.
2. Copia el documento de este paso y estudia sus apartados. Relaciónalo con los bloques de teoría del punto 3.2; no debe limitarse a una lista de nombres.
3. Distingue los componentes del catálogo arquitectónico de las clases y los servicios implementados en el proyecto.
4. Guarda el Markdown, abre su vista previa y anota dos decisiones observables que podrás verificar posteriormente con los tests.

Archivo que debe quedar tras este paso: `docs/puertos-adaptadores.md`

```
# Puertos y adaptadores — Punto 3.2

## Puerto

Interfaz que define cómo el núcleo se comunica con el exterior. Se define en el núcleo.

## Adaptador

Clase que implementa un puerto y conecta el núcleo con una tecnología concreta. Se define en el exterior.

## Tipos de puerto

- Puerto de entrada: lo que el núcleo ofrece al exterior.
- Puerto de salida: lo que el núcleo necesita del exterior.

## Tipos de adaptador

- Adaptador de entrada: traduce entre el protocolo y los puertos de entrada.
- Adaptador de salida: traduce entre los puertos de salida y la tecnología concreta.
```

```
## Puertos del proyecto

| Puerto | Tipo | Ubicación |
|--------|------|-----------|
| IOrdenServicio | Entrada | Aplicación |
| IInventarioServicio | Entrada | Aplicación |
| IOrdenRepositorio | Salida | Aplicación |
| IInventarioRepositorio | Salida | Aplicación |
| IPublicadorEventos | Salida | Aplicación |

## Adaptadores del proyecto

| Adaptador | Tipo | Ubicación |
|-----------|------|-----------|
| Endpoints de Minimal API | Entrada | Presentación |
| OrdenRepositorioSqlServer | Salida | Infraestructura |
| InventarioRepositorioSqlServer | Salida | Infraestructura |
| PublicadorEventosRabbitMq | Salida | Infraestructura |

## Inversión de dependencias

El núcleo depende del puerto. El adaptador implementa el puerto. El contenedor inyecta la implementación.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Puertos de entrada | Interfaces de casos de uso |
| Puertos de salida | Interfaces de recursos externos |
| Adaptadores entrada | Endpoints de Minimal API |
| Adaptadores salida | Repositorios SQL Server, publicador de eventos |
| Inversión | Registro en el contenedor de dependencias |

## Corrección de compatibilidad con el repositorio vigente

La implementación de partida no es `AceriaApp.Monolito.Api`, sino la Minimal API del proyecto
`M02_APIS_REST_ASPNET_CORE/PROYECTO/2.4-documentacion-openapi`. Los ejemplos de esta ficha describen
conceptos de diseño; su implementación real se incorpora al código en 3.3 y 3.4, con puertos de aplicación e
infraestructura SQL Server. No se considera registrado un adaptador solo por mencionarlo en un catálogo.
```

Verificación concreta. el archivo `docs\puertos-adaptadores.md` existe con las ocho secciones.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 3.2.5 — Crear la carpeta `PuertosAdaptadores` en el proyecto `AceriaApp.Api`

La carpeta `PuertosAdaptadores` albergará las clases de validación de puertos y adaptadores. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el Explorador de soluciones, dentro del proyecto `AceriaApp.Api`, crea la carpeta `PuertosAdaptadores` si no existe.
2. Comprueba que el directorio físico es `src/AceriaApp.Api/PuertosAdaptadores` y no está situado dentro del proyecto de tests.
3. Prepara los tres archivos de este punto: `Puerto.cs`, `ValidadorPuertosAdaptadores.cs` y `CatalogoPuertosAdaptadores.cs`, que añadirás en los siguientes pasos.
4. No confundas estos modelos de catálogo arquitectónico con la implementación real del puerto de órdenes, que aparecerá más adelante.

Verificación concreta. la carpeta `PuertosAdaptadores` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 3.2.6 — Crear la clase `Puerto`

`Puerto` expresa un contrato desde el punto de vista del núcleo. Una instancia de esta clase es todavía una descripción, no una interfaz de aplicación registrada en DI.

1. Crea `src/AceriaApp.Api/PuertosAdaptadores/Puerto.cs` y escribe el código íntegro mostrado en el bloque asociado al paso.
2. Comprueba que el namespace es `AceriaApp.Api.PuertosAdaptadores`, correspondiente a la carpeta de este punto.
3. Lee cada propiedad y diferencia valores de configuración de objetos de dominio. No añadas dependencias SQL o ASP.NET Core a este modelo de catálogo.
4. Guarda y compila con `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`; si faltan tipos, revisa el namespace y los usings implícitos.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/PuertosAdaptadores/Puerto.cs`

```
namespace AceriaApp.Api.PuertosAdaptadores;

public class Puerto
{
    public string Nombre { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;
    public string Capa { get; init; } = string.Empty;
}
```

Verificación concreta. la clase `Puerto` compila sin errores y expone tres propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 3.2.7 — Crear la clase `Adaptador`

`Adaptador` describe quién puede satisfacer un puerto y de qué modo. La clase de catálogo permite estudiar la idea; la implementación ejecutable del adaptador llegará con la composición real.

1. Crea `src/AceriaApp.Api/PuertosAdaptadores/Adaptador.cs` y copia la definición de la clase del bloque de este paso.
2. Comprueba que identifica el adaptador, su tipo, el puerto implementado y la capa donde reside.
3. Distingue adaptadores de entrada y salida y evita acoplar el núcleo a una clase HTTP concreta.
4. Guarda y compila; el validador de correspondencia aparecerá en el paso 8.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/PuertosAdaptadores/Adaptador.cs`

```
namespace AceriaApp.Api.PuertosAdaptadores;

public class Adaptador
{
    public string Nombre { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;
    public string PuertoImplementado { get; init; } = string.Empty;
```

```
    public string Capa { get; init; } = string.Empty;
}
```

Verificación concreta. la clase `Adaptador` compila sin errores y expone cuatro propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 3.2.8 — Crear la clase `ValidadorPuertosAdaptadores`

El validador comprueba asociaciones de puertos y adaptadores según reglas explícitas. Debe ser posible relacionar cada rechazo con una regla y una prueba, sin deducir que el sistema ya está conectado.

1. Crea `src/AceriaApp.Api/PuertosAdaptadores/ValidadorPuertosAdaptadores.cs` y copia el validador de los puertos y adaptadores.
2. Sigue las ramas que comprueban tipo de puerto, capa de definición, existencia del puerto y ubicación del adaptador.
3. Comprueba que las colecciones del catálogo se comparan por el nombre del puerto, no por identidad de objeto.
4. Guarda y compila; prepara una prueba con adaptador vinculado a un puerto inexistente.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/PuertosAdaptadores/ValidadorPuertosAdaptadores.cs`

```
namespace AceriaApp.Api.PuertosAdaptadores;

public class ValidadorPuertosAdaptadores
{
    private readonly List<Puerto> _puertos;
    private readonly List<Adaptador> _adaptadores;

    public ValidadorPuertosAdaptadores(List<Puerto> puertos, List<Adaptador> adaptadores)
    {
        _puertos = puertos;
        _adaptadores = adaptadores;
    }

    public ResultadoValidacionHexagonal Validar()
    {
        var errores = new List<string>();

        foreach (var puerto in _puertos)
        {
            if (puerto.Tipo != "Entrada" && puerto.Tipo != "Salida")
            {
                errores.Add($"Tipo de puerto no válido: '{puerto.Tipo}'.");
            }

            if (puerto.Capa != "Aplicacion")
            {
                errores.Add($"El puerto '{puerto.Nombre}' no está en la capa de aplicación.");
            }
        }

        foreach (var adaptador in _adaptadores)
        {
            var puerto = _puertos.FirstOrDefault(p => p.Nombre == adaptador.PuertoImplementado);
            if (puerto is null)
            {
                errores.Add($"El adaptador '{adaptador.Nombre}' implementa un puerto desconocido:
'{adaptador.PuertoImplementado}'.");
                continue;
            }
```

```
            if (puerto.Tipo == "Entrada" && adaptador.Capa != "Presentacion")
            {
                errores.Add($"El adaptador de entrada '{adaptador.Nombre}' no está en la capa de
presentación.");
            }

            if (puerto.Tipo == "Salida" && adaptador.Capa != "Infraestructura")
            {
                errores.Add($"El adaptador de salida '{adaptador.Nombre}' no está en la capa de
infraestructura.");
            }
        }

        return new ResultadoValidacionHexagonal(errores.Count == 0, errores);
    }
}

public record ResultadoValidacionHexagonal(bool EsValida, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorPuertosAdaptadores` compila sin errores y expone un constructor, un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` y `IReadOnlyList<T>` no se resuelven.
- Olvidar el `using System.Linq;`: `FirstOrDefault` no se resuelve.
- No comprobar si el puerto del adaptador existe: lanza `NullReferenceException`.

PASO 3.2.9 — Añadir los `using` necesarios en `ValidadorPuertosAdaptadores`

Los tipos `List<T>` y `IReadOnlyList<T>` pertenecen a `System.Collections.Generic`. El método `FirstOrDefault` pertenece a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. Crea `src/AceriaApp.Api/PuertosAdaptadores/CatalogoPuertosAdaptadores.cs` con el catálogo y los valores de ejemplo indicados en el código del laboratorio.
2. Identifica las reglas o registros que el catálogo declara y cuáles pertenecen únicamente al ejercicio conceptual; no los des por registrados en DI.
3. Comprueba que el catálogo utiliza los nombres de capas, puertos o proyectos acordados y que no declara interfaces imposibles de resolver en tiempo de compilación.
4. Guarda y compila la API. Relaciona cada ejemplo con una prueba del paso 15.

Verificación concreta. `ValidadorPuertosAdaptadores.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `FirstOrDefault` no se resuelve.
- Añadir los usings después del namespace: el compilador no los reconoce.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 3.2.10 — Crear la clase `CatalogoPuertosAdaptadores`

`CatalogoPuertosAdaptadores` ayuda a analizar distintas propuestas. Su resultado no añade servicios a ASP.NET Core ni cambia las referencias entre ensamblados.

1. Abre `src/AceriaApp.Api/PuertosAdaptadores/CatalogoPuertosAdaptadores.cs` y observa las colecciones e importaciones utilizadas por el catálogo.
2. Revisa `using System;`, `using System.Linq;` y `using System.Collections.Generic;` cuando corresponda; conserva solo lo que la compilación necesite.

3. No añadas dependencias tecnológicas nuevas al dominio para resolver tipos básicos ni vuelvas a crear el archivo de catálogo.
4. Compila y confirma que los métodos del catálogo pueden llamarse desde el proyecto de tests sin errores de namespace.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/PuertosAdaptadores/CatalogoPuertosAdaptadores.cs`

```
namespace AceriaApp.Api.PuertosAdaptadores;

public class CatalogoPuertosAdaptadores
{
    public static List<Puerto> PuertosPorDefecto()
    {
        return new List<Puerto>
        {
            new() { Nombre = "IOrdenServicio", Tipo = "Entrada", Capa = "Aplicacion" },
            new() { Nombre = "IInventarioServicio", Tipo = "Entrada", Capa = "Aplicacion" },
            new() { Nombre = "IOrdenRepositorio", Tipo = "Salida", Capa = "Aplicacion" },
            new() { Nombre = "IInventarioRepositorio", Tipo = "Salida", Capa = "Aplicacion" },
            new() { Nombre = "IPublicadorEventos", Tipo = "Salida", Capa = "Aplicacion" }
        };
    }

    public static List<Adaptador> AdaptadoresPorDefecto()
    {
        return new List<Adaptador>
        {
            new() { Nombre = "EndpointsMinimalApi", Tipo = "Entrada", PuertoImplementado = "IOrdenServicio",
Capa = "Presentacion" },
            new() { Nombre = "OrdenRepositorioSqlServer", Tipo = "Salida", PuertoImplementado =
"IOrdenRepositorio", Capa = "Infraestructura" },
            new() { Nombre = "PublicadorEventosRabbitMq", Tipo = "Salida", PuertoImplementado =
"IPublicadorEventos", Capa = "Infraestructura" }
        };
    }
}
```

Verificación concreta. la clase `CatalogoPuertosAdaptadores` compila sin errores y expone dos métodos estáticos.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Devolver `List<T>` mutable: expone la implementación interna.

PASO 3.2.11 — Crear el proyecto de tests del punto 3.2

El punto 3.2 añade un proyecto de tests específico para las clases de puertos y adaptadores. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Dentro de `tests` crea el proyecto xUnit `AceriaApp.PuertosAdaptadores.Tests` con destino `net8.0`. No modifiques ni sustituyas `AceriaApp.Api.Tests`, que contiene las pruebas HTTP heredadas de M02.
2. Crea el archivo `tests/AceriaApp.PuertosAdaptadores.Tests/AceriaApp.PuertosAdaptadores.Tests.csproj` y escribe el XML completo del bloque del laboratorio, con xUnit y FluentAssertions 6.12.2.
3. Comprueba que incluye un `ProjectReference` hacia `../../src/AceriaApp.Api/AceriaApp.Api.csproj`; agrega el nuevo proyecto con `dotnet sln AceriaApp.sln add tests/AceriaApp.PuertosAdaptadores.Tests/AceriaApp.PuertosAdaptadores.Tests.csproj`.
4. Ejecuta `dotnet restore AceriaApp.sln` y comprueba que no se ha generado un test `UnitTest1` no previsto.

Archivo que debe quedar tras este paso: `tests/AceriaApp.PuertosAdaptadores.Tests/AceriaApp.PuertosAdaptadores.Tests.csproj`

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</ImplicitUsings><IsTestProject>true</IsTestProject><IsPackable>false</IsPackable></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El proyecto de tests `AceriaApp.PuertosAdaptadores.Tests` restaura, referencia a `AceriaApp.Api` y no sustituye ninguna suite heredada.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 3.1: viola la regla de no modificar archivos del punto anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 3.2.12 — Añadir la referencia del proyecto de tests a la API `AceriaApp.Api`

La referencia permite que el proyecto de tests acceda a las clases de `PuertosAdaptadores` del proyecto `AceriaApp.Api`.

1. En `tests/AceriaApp.PuertosAdaptadores.Tests/AceriaApp.PuertosAdaptadores.Tests.csproj` localiza la referencia al proyecto `AceriaApp.Api`.
2. Verifica la dirección de la dependencia: los tests referencian a la API, nunca la API al proyecto de tests.
3. Ejecuta `dotnet sln AceriaApp.sln list` y verifica que el nuevo proyecto aparece junto a los heredados, con la ruta física correcta.
4. No dupliques `ProjectReference` si la referencia ya está preparada en el XML del paso 11.

Verificación concreta. El proyecto de tests `AceriaApp.PuertosAdaptadores.Tests` restaura, referencia a `AceriaApp.Api` y no sustituye ninguna suite heredada.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 3.2.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Abre el `.csproj` de `AceriaApp.PuertosAdaptadores.Tests` y localiza el `PackageReference` a `FluentAssertions` versión 6.12.2.
2. Ejecuta `dotnet list tests/AceriaApp.PuertosAdaptadores.Tests/AceriaApp.PuertosAdaptadores.Tests.csproj package` y comprueba que el paquete consta en el proyecto correcto.
3. Relaciona el método `Should()` de los ejemplos de tests con FluentAssertions, y `[Fact]` con xUnit.
4. No instales una segunda versión del mismo paquete: basta con restaurar NuGet desde la solución.

Verificación concreta. El proyecto de tests `AceriaApp.PuertosAdaptadores.Tests` restaura, referencia a `AceriaApp.Api` y no sustituye ninguna suite heredada.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 3.2.14 — Crear el archivo `ValidadorPuertosAdaptadoresTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza puertos fuera de la capa de aplicación, adaptadores de puertos desconocidos y adaptadores mal ubicados.

1. Crea `tests/AceriaApp.PuertosAdaptadores.Tests/ValidadorPuertosAdaptadoresTests.cs` y escribe completo el primer bloque de pruebas de este punto.
2. Revisa el `using AceriaApp.Api.PuertosAdaptadores` para que coincida con el namespace actualizado y conserva `using Xunit;` y `using FluentAssertions;`.
3. Localiza los casos correctos e incorrectos del validador y comprueba que las aserciones describen resultados de código ejecutable.
4. Guarda, compila y ejecuta el test específico. No cambies las expectativas para ocultar una regla equivocada.

Archivo que debe quedar tras este paso: `tests/AceriaApp.PuertosAdaptadores.Tests/ValidadorPuertosAdaptadoresTests.cs`

```
using AceriaApp.Api.PuertosAdaptadores;
using FluentAssertions;
using Xunit;

namespace AceriaApp.PuertosAdaptadores.Tests;

public class ValidadorPuertosAdaptadoresTests
{
    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var validador = new ValidadorPuertosAdaptadores(
            CatalogoPuertosAdaptadores.PuertosPorDefecto(),
            CatalogoPuertosAdaptadores.AdaptadoresPorDefecto());

        var resultado = validador.Validar();

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_PuertoFueraDeAplicacion_DevuelveError()
    {
        var puertos = new List<Puerto>
        {
            new() { Nombre = "IPuertoMal", Tipo = "Entrada", Capa = "Infraestructura" }
        };
        var validador = new ValidadorPuertosAdaptadores(puertos, new List<Adaptador>());

        var resultado = validador.Validar();

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no está en la capa de aplicación"));
    }

    [Fact]
    public void Validar_AdaptadorDePuertoDesconocido_DevuelveError()
    {
        var adaptadores = new List<Adaptador>
        {
```

```
            new() { Nombre = "AdaptadorHuerfano", Tipo = "Salida", PuertoImplementado = "IPuertoInexistente",
Capa = "Infraestructura" }
        };
        var validador = new ValidadorPuertosAdaptadores(new List<Puerto>(), adaptadores);

        var resultado = validador.Validar();

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("puerto desconocido"));
    }

    [Fact]
    public void Validar_AdaptadorDeEntradaFueraDePresentacion_DevuelveError()
    {
        var puertos = new List<Puerto>
        {
            new() { Nombre = "IOrdenServicio", Tipo = "Entrada", Capa = "Aplicacion" }
        };
        var adaptadores = new List<Adaptador>
        {
            new() { Nombre = "AdaptadorMalUbicado", Tipo = "Entrada", PuertoImplementado = "IOrdenServicio",
Capa = "Infraestructura" }
        };
        var validador = new ValidadorPuertosAdaptadores(puertos, adaptadores);

        var resultado = validador.Validar();

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no está en la capa de presentación"));
    }

    [Fact]
    public void Validar_AdaptadorDeSalidaFueraDeInfraestructura_DevuelveError()
    {
        var puertos = new List<Puerto>
        {
            new() { Nombre = "IOrdenRepositorio", Tipo = "Salida", Capa = "Aplicacion" }
        };
        var adaptadores = new List<Adaptador>
        {
            new() { Nombre = "AdaptadorMalUbicado", Tipo = "Salida", PuertoImplementado =
"IOrdenRepositorio", Capa = "Presentacion" }
        };
        var validador = new ValidadorPuertosAdaptadores(puertos, adaptadores);

        var resultado = validador.Validar();

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no está en la capa de infraestructura"));
    }
}
```

Verificación concreta. Los tests de `AceriaApp.PuertosAdaptadores.Tests` compilan y ejecutan las aserciones propuestas para `PuertosAdaptadores`.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.PuertosAdaptadores;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso del catálogo por defecto: el validador no se prueba en el caso feliz.

PASO 3.2.15 — Crear el archivo `CatalogoPuertosAdaptadoresTests.cs`

Los tests verifican que el catálogo por defecto contiene los puertos y adaptadores esperados.

1. Crea `tests/AceriaApp.PuertosAdaptadores.Tests/CatalogoPuertosAdaptadoresTests.cs` y escribe el segundo bloque de pruebas del catálogo.
2. Comprueba que las llamadas a los métodos de ejemplo de `{catalog}` son compatibles con las propiedades que realmente expone el código.

3. Guarda y ejecuta `dotnet test tests/AceriaApp.PuertosAdaptadores.Tests/AceriaApp.PuertosAdaptadores.Tests.csproj -c Release`, de forma que se ejecuten todas las pruebas nuevas.
4. Verifica que las aserciones prueban los valores declarados, pero no confunde un catálogo didáctico con un registro en tiempo de ejecución.

Archivo que debe quedar tras este paso: `tests/AceriaApp.PuertosAdaptadores.Tests/CatalogoPuertosAdaptadoresTests.cs`

```
using AceriaApp.Api.PuertosAdaptadores;
using FluentAssertions;
using Xunit;

namespace AceriaApp.PuertosAdaptadores.Tests;

public class CatalogoPuertosAdaptadoresTests
{
    [Fact]
    public void PuertosPorDefecto_ContieneCincoPuertos()
    {
        var puertos = CatalogoPuertosAdaptadores.PuertosPorDefecto();

        puertos.Should().HaveCount(5);
    }

    [Fact]
    public void PuertosPorDefecto_ContieneDosEntradasYTresSalidas()
    {
        var puertos = CatalogoPuertosAdaptadores.PuertosPorDefecto();

        puertos.Count(p => p.Tipo == "Entrada").Should().Be(2);
        puertos.Count(p => p.Tipo == "Salida").Should().Be(3);
    }

    [Fact]
    public void AdaptadoresPorDefecto_ContieneTresAdaptadores()
    {
        var adaptadores = CatalogoPuertosAdaptadores.AdaptadoresPorDefecto();

        adaptadores.Should().HaveCount(3);
    }
}
```

Verificación concreta. Los tests de `AceriaApp.PuertosAdaptadores.Tests` compilan y ejecutan las aserciones propuestas para `PuertosAdaptadores`.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.PuertosAdaptadores;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Count` no se resuelve.
- No verificar la distribución de tipos: el catálogo podría estar mal balanceado.

PASO 3.2.16 — Compilar la solución completa

Compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

1. Abre la terminal junto a `AceriaApp.sln` y ejecuta `dotnet build AceriaApp.sln -c Release` después de introducir todos los archivos del punto.
2. Comprueba que compilan la API de M02, el proyecto nuevo de tests y todos los proyectos de etapas anteriores.
3. Si aparece un error, identifica el primer archivo y la primera línea antes de tocar otra clase. Revisa referencias y namespaces.
4. Registra la salida de la compilación y no avances hasta obtener cero errores.

Verificación concreta. La solución completa compila en Release con cero errores. La capa de dominio no referencia EF Core ni ASP.NET Core.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 3.2.17 — Ejecutar todos los tests del punto 3.2

Ejecutar los tests del punto 3.2 verifica que las clases de puertos y adaptadores funcionan correctamente.

1. Ejecuta `dotnet test tests/AceriaApp.PuertosAdaptadores.Tests/AceriaApp.PuertosAdaptadores.Tests.csproj -c Release` desde la raíz del punto.
2. Comprueba que los métodos `[Fact]` de `ValidadorPuertosAdaptadoresTests` y `CatalogoPuertosAdaptadoresTests` pasan; el resumen numérico exacto aparece en la tabla de resultados de este módulo.
3. Ejecuta un caso inválido del validador en el depurador y relaciona el mensaje de error con la línea que lo produce.
4. Conserva la salida de tests como evidencia y confirma que los resultados no dependen de SQL Server para evaluar reglas puras.

Verificación concreta. La suite nueva de 3.2 supera 8 pruebas de catálogo; en 3.4 se añaden además 4 pruebas HTTP del adaptador real.

Diagnóstico si falla.

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 3.2.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 3.2 pasan conjuntamente.

1. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después de compilar toda la solución en Release.
2. Comprueba que pasan los tests de M02 y todos los nuevos del módulo M03 acumulados hasta este punto.
3. Si falla alguna suite previa, vuelve a comprobar si el último cambio alteró una dependencia o una configuración del servidor.
4. Registra las pruebas totales, fallidas y omitidas y compáralas con el resultado previsto para esta etapa.

Verificación concreta. Pruebas acumuladas previstas para 3.2: 31, 0 fallidas. Comprueba estos valores en la salida de `dotnet test`.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.

- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

PASO 3.2.19 — Crear el README del punto 3.2

El README del punto 3.2 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 3.1.

1. Abre `README.md` en la raíz del punto `3.2-puertos-adaptadores` y sustitúyelo por el README específico del bloque de este paso.
2. Comprueba que los nombres de proyectos y las pruebas acumuladas coinciden con la solución y con la salida de `dotnet test` de esta etapa.
3. Revisa la ubicación del documento `docs` y los comandos `dotnet build` y `dotnet test` que utilizará el alumno.
4. Guarda la versión del punto actual sin sobrescribir el README de la etapa anterior.

Archivo que debe quedar tras este paso: `README.md`

```
# Punto 3.2 — Carpeta del punto 3.2 a partir del punto 3.1

**Curso:** Microservicios y arquitectura hexagonal con ASP.NET Core 8
**Código de referencia:** AceriaApp, en continuidad con M02 2.4.

## Objetivo

el documento de puertos y adaptadores es el artefacto de `docs` específico del punto 3.2. No modifica los
catorce documentos existentes del punto 3.1.

## Estado verificado del punto

- 4 proyectos en la solución `AceriaApp.sln` al finalizar este punto.
- 31 pruebas xUnit acumuladas previstas (incluyendo las 16 de M02 y las nuevas del módulo 3).
- Clases del catálogo `PuertosAdaptadores` en `src/AceriaApp.Api/PuertosAdaptadores`.
- Documentación `docs/puertos-adaptadores.md` y decisión `docs/ADR-011-puertos-adaptadores.md`.
- Los endpoints REST del módulo 2 se conservan sin cambios funcionales.

## Cómo ejecutarlo

1. Abrir el archivo `AceriaApp.sln` de esta carpeta en Visual Studio, con SDK .NET 8.
2. Ejecutar `dotnet restore AceriaApp.sln` y después `dotnet build AceriaApp.sln -c Release`.
3. Ejecutar `dotnet test AceriaApp.sln -c Release --no-build`. La suite M03 se construye por adición desde
M02.
4. La aplicación principal utiliza SQL Server Express LocalDB; los tests HTTP mantienen SQL Server y crean
bases exclusivas independientes mediante `BaseSqlServer.Crear()`.
5. En 3.4, los endpoints hexagonales reales están en `/api/v1/hexagonal/ordenes` y utilizan un repositorio EF
Core a través de un puerto de aplicación.

## Qué se ha añadido

la clase `Puerto` modela un puerto con su nombre, tipo y capa. Es la unidad de información del validador.
```

Verificación concreta. `README.md` describe correctamente los 4 proyectos y 31 pruebas acumuladas del curso vigente.

Diagnóstico si falla.

- Modificar el README del punto 3.1: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 3.1: rompe la separación.

PASO 3.2.20 — Crear el ADR-011 del punto 3.2

El ADR-011 documenta la decisión de adoptar puertos y adaptadores. Es un artefacto nuevo del punto 3.2 que no modifica los ADR existentes.

1. En la carpeta `docs` crea `ADR-011-puertos-adaptadores.md` y copia el documento de decisión del bloque del laboratorio.
2. Lee Estado, Contexto, Decisión, Alternativas y Consecuencias y relaciona el ADR con el código nuevo de la etapa.
3. Explica por qué no basta con declarar nombres de capas si el código puede saltarse la interfaz o registrar una dependencia incorrecta.
4. Comprueba que los ADR anteriores están presentes y no han sido sobrescritos.

Archivo que debe quedar tras este paso: `docs/ADR-011-puertos-adaptadores.md`

```
# ADR-011: Puertos y adaptadores

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado la organización por capas y la regla de dependencia. Es necesario definir
los puertos y adaptadores que conectan el núcleo con el exterior.

## Decisión
Se adopta la arquitectura hexagonal con puertos de entrada y de salida. Los puertos se definen en la capa de
aplicación. Los adaptadores de entrada se sitúan en la capa de presentación. Los adaptadores de salida se
sitúan en la capa de infraestructura. La inversión de dependencias se aplica con el contenedor de
dependencias. La validación de la coherencia se realiza con un validador automático.

## Alternativas consideradas
- Acoplar el núcleo a los adaptadores: descartado por violar la regla de dependencia.
- Definir un solo tipo de puerto: descartado por pérdida de información.
- Validar manualmente la coherencia: descartado por falta de trazabilidad.

## Consecuencias
- Positivas: núcleo independiente, adaptadores intercambiables, validación automática.
- Negativas: la definición de puertos requiere disciplina.
- Mitigación: validación automática con el validador.

## Aplicación en AceriaApp

Se parte de la API real y probada de M02, se conservan sus endpoints y se añaden reglas de dependencia
verificadas en código. Los proyectos de Dominio, Aplicación e Infraestructura deben obedecer la dirección de
referencias; en 3.4 se expone un nuevo adaptador HTTP funcional. El catálogo describe las relaciones
arquitectónicas; cada implementación se comprueba en su proyecto correspondiente.
```

Verificación concreta. el archivo `docs\ADR-011-puertos-adaptadores.md` existe con las cinco secciones.

Diagnóstico si falla.

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-011`: rompe la convención.

PASO 3.2.21 — Verificar que no se ha modificado ningún archivo del punto 3.1

La comparación con `fc` verifica que todos los documentos del punto 3.1 no se han modificado.

1. Localiza el mismo documento heredado tanto en `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.1-capas-aplicacion` como en `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.2-puertos-adaptadores`.
2. Compara sus hashes SHA-256 o usa un diff y confirma que no se reescribió al introducir el nuevo punto.

3. Distingue de él los archivos añadidos en este punto: catálogo, validador, pruebas y documentación nueva.
4. Documenta cualquier diferencia inesperada; el nuevo código no exige alterar los archivos de etapas anteriores.

Verificación concreta. Los archivos heredados de `docs` y los endpoints M02 no han sido sobrescritos.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Olvidar el flag `/L`: `fc` compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 3.2.22 — Verificar la estructura del punto 3.2

La verificación estructural confirma que el punto 3.2 añade archivos nuevos sin modificar los existentes.

1. Desde la raíz de `M03_ARQUITECTURA_HEXAGONAL/PROYECTO` ejecuta `tree /F 3.2-puertos-adaptadores` en Windows.
2. Localiza `src/AceriaApp.Api/PuertosAdaptadores`, `tests/AceriaApp.PuertosAdaptadores.Tests`, `README.md` y `docs/puertos-adaptadores.md`.
3. Comprueba los proyectos adicionales de dominio, aplicación e infraestructura si ya fueron introducidos por el punto 3.3.
4. Distingue los archivos del proyecto de los temporales `obj/` y `bin/`; ambos deben quedar excluidos de la entrega.

Verificación concreta. Los archivos que aparecen en el árbol corresponden a los fragmentos creados hasta este paso, y no hay código pendiente en carpetas fuera de la solución.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 3.1: viola la regla de no modificar.
- Omitir la carpeta `PuertosAdaptadores`: los tests no encuentran las clases.

PASO 3.2.23 — Verificar el `.gitignore` del punto 3.2

El `.gitignore` se hereda del punto 3.1 sin modificaciones. La verificación confirma que no se ha tocado.

1. Abre el `.gitignore` de la etapa; si falta, crea uno que excluya `.vs/`, `bin/`, `obj/`, archivos personales y resultados de prueba.
2. Ejecuta `git status --short` en tu copia de trabajo y comprueba que no aparecen archivos binarios ni credenciales.
3. No ignores `src`, `tests`, `docs` ni las soluciones `.sln`, pues son parte del entregable reproducible.
4. Conserva el `.gitignore` heredado sin borrarlo ni modificar el de etapas anteriores.

Verificación concreta. No se incluyen `bin`, `obj`, contraseñas ni resultados de pruebas en el estado Git.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 3.1: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 3.2.24 — Hacer el commit del punto 3.2 y verificar el estado final

El commit del punto 3.2 registra el estado acumulado del proyecto. Los commits de cada punto permiten comparar el estado del proyecto entre puntos consecutivos y entre módulos completos.

1. Antes de terminar 3.2, vuelve a ejecutar `dotnet build AceriaApp.sln -c Release` y `dotnet test AceriaApp.sln -c Release --no-build`.
2. Lee el reto de ampliación incluido tras los pasos obligatorios; si lo realizas, utiliza una copia aparte y deja los tests base intactos.
3. Si tu copia está bajo Git, ejecuta `git status`, añade solo lo trabajado y crea un commit descriptivo, sin modificar la rama canónica del profesor.
4. Conserva el resumen de pruebas y compara el proyecto final con la carpeta de referencia del punto; ambas deben representar el mismo código.

Verificación concreta. El punto termina con compilación y suite de pruebas completa, sin modificar la referencia de la etapa anterior.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Esta ampliación añade una operación de resumen al validador y una prueba de su resultado. **Realízala después de superar la etapa básica y sus tests.** Incorpora cada fragmento en la clase indicada y conserva el namespace del proyecto en el que trabajas.

Reto: añadir un método `Resumen` a `ValidadorPuertosAdaptadores` que devuelva el número de puertos de entrada, el número de puertos de salida y el número de adaptadores, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorPuertosAdaptadores.cs`, añadir el siguiente `record` al final del archivo:

```
public record ResumenHexagonal(int PuertosEntrada, int PuertosSalida, int Adaptadores);
```

Paso 2: Explicar línea por línea:

- **Línea 1:** `public record ResumenHexagonal(` → declara un contrato inmutable.
- **Línea 2:** `int PuertosEntrada,` → número de puertos de entrada.
- **Línea 3:** `int PuertosSalida,` → número de puertos de salida.
- **Línea 4:** `int Adaptadores);` → número de adaptadores.

Paso 3: En `ValidadorPuertosAdaptadores.cs`, añadir el siguiente método dentro de la clase:

```
public ResumenHexagonal Resumen()
{
    var entradas = _puertos.Count(p => p.Tipo == "Entrada");
    var salidas = _puertos.Count(p => p.Tipo == "Salida");
    var adaptadores = _adaptadores.Count;
    return new ResumenHexagonal(entradas, salidas, adaptadores);
}
```

Paso 4: Explicar línea por línea:

- **Línea 1:** `public ResumenHexagonal Resumen()` → declara el método.
- **Línea 2:** `{` → abre el cuerpo.

- **Línea 3:** `var entradas = _puertos.Count(p => p.Tipo == "Entrada");` → cuenta los puertos de entrada.
- **Línea 4:** `var salidas = _puertos.Count(p => p.Tipo == "Salida");` → cuenta los puertos de salida.
- **Línea 5:** `var adaptadores = _adaptadores.Count;` → cuenta los adaptadores.
- **Línea 6:** `return new ResumenHexagonal(entradas, salidas, adaptadores);` → devuelve el resumen.
- **Línea 7:** `}` → cierra el cuerpo.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorPuertosAdaptadoresTests.cs`, añadir el siguiente test:

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CuentaCorrectamente()
{
    var validador = new ValidadorPuertosAdaptadores(
        CatalogoPuertosAdaptadores.PuertosPorDefecto(),
        CatalogoPuertosAdaptadores.AdaptadoresPorDefecto());

    var resumen = validador.Resumen();

    resumen.PuertosEntrada.Should().Be(2);
    resumen.PuertosSalida.Should().Be(3);
    resumen.Adaptadores.Should().Be(3);
}
```

Paso 7: Explicar línea por línea:

- **Línea 1:** `[Fact]` → marca el método como test de xUnit.
- **Línea 2:** `public void Resumen_ConCatalogoPorDefecto_CuentaCorrectamente()` → declara el test.
- **Línea 3:** `{` → abre el cuerpo.
- **Línea 4:** `var validador = new ValidadorPuertosAdaptadores(` → instancia el validador.
- **Línea 5:** `CatalogoPuertosAdaptadores.PuertosPorDefecto(),` → carga los puertos.
- **Línea 6:** `CatalogoPuertosAdaptadores.AdaptadoresPorDefecto());` → carga los adaptadores.
- **Línea 7:** `var resumen = validador.Resumen();` → invoca el método.
- **Línea 8:** `resumen.PuertosEntrada.Should().Be(2);` → verifica los puertos de entrada.
- **Línea 9:** `resumen.PuertosSalida.Should().Be(3);` → verifica los puertos de salida.
- **Línea 10:** `resumen.Adaptadores.Should().Be(3);` → verifica los adaptadores.
- **Línea 11:** `}` → cierra el cuerpo.

Paso 8: Ejecutar `dotnet test tests\AceriaApp.PuertosAdaptadores.Tests\AceriaApp.PuertosAdaptadores.Tests.csproj` y verificar que pasan los nueve tests.

Paso 9: Verificar que el método `Resumen` no modifica ningún archivo del punto 3.1.

Paso 10: Hacer commit con `git commit -m "Reto 3.2: método Resumen en ValidadorPuertosAdaptadores"`.

Verificación: la salida de `dotnet test` muestra `Passed: 9`.

Resultado verificable y cierre del punto 3.2

Código final de referencia: `PROYECTO/3.2-puertos-adaptadores/`. El contenido que debe crear el alumno está especificado por los bloques con su ruta exacta y el tipo de modificación. Comprueba el resultado compilando y ejecutando las pruebas de la etapa.

Continuidad: las dependencias existentes y la documentación heredada se conservan; el avance del punto 3.2 prepara el siguiente sin reescribir la etapa anterior.

Punto 3.3 — Separación entre dominio, aplicación e infraestructura

Hacer visibles las dependencias entre dominio, aplicación e infraestructura

La etapa 3.3 convierte la reflexión arquitectónica en una separación que pueda observarse mediante proyectos y referencias .NET. El dominio contiene las reglas; la aplicación coordina casos de uso y define puertos; la infraestructura implementa colaboraciones externas sin hacer que las capas interiores dependan de EF Core.

Comprobaremos las referencias reales y la compilación, no solo el resultado de los validadores de catálogo. Un proyecto con nombre «Dominio» no queda protegido de dependencias indebidas si importa paquetes de infraestructura. El alumno deberá distinguir las reglas modeladas por las clases docentes de las reglas que el compilador puede verificar mediante ProjectReference.

Objetivos del punto. el documento de separación de capas es el artefacto de `docs` específico del punto 3.3. No modifica los quince documentos existentes del punto 3.2.

Carpeta de trabajo: `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.3-separacion-capas`. Esta etapa continúa la anterior sin alterar su copia. El código de la referencia es un resultado final, no una solución que el alumno deba pegar al inicio.

Criterio de aceptación: seguir los 24 pasos, compilar y ejecutar los tests; comprobar los resultados indicados y registrar las actividades manuales. Al finalizar, la solución debe reproducir el comportamiento descrito.

PASO 3.3.1 — Crear la carpeta del punto 3.3 a partir del punto 3.2

Cada punto del curso parte del contenido del punto anterior. El punto 3.3 se construye sobre el punto 3.2 copiando su contenido y añadiendo archivos nuevos sin modificar los existentes.

1. En el Explorador de archivos localiza `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.2-puertos-adaptadores`. Comprueba que es el estado anterior completo, con los endpoints y tests ya construidos.
2. Crea una carpeta personal `3.3-separacion-capas` bajo `M03_ARQUITECTURA_HEXAGONAL/PROYECTO`. Copia en ella el contenido del punto anterior, sin cambiarlo en su origen.
3. Desde la raíz de `3.3-separacion-capas` comprueba que tienes `src/AceriaApp.Api`, `tests/AceriaApp.Api.Tests`, `AceriaApp.http` y los archivos de configuración.
4. No copies `bin/`, `obj/` ni credenciales. Para comparar puedes usar los proyectos de referencia; los nuevos archivos de este punto se introducirán solo cuando llegues a su paso.

Verificación concreta. al ejecutar `dir 3.3-separacion-capas`, deben aparecer `AceriaApp.sln`, `README.md`, `src`, `tests` y `docs`.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

PASO 3.3.2 — Abrir la solución copiada en Visual Studio 2026

Abrir la solución copiada permite trabajar en el punto 3.3 sobre la base del punto 3.2. La compilación verifica que la copia es correcta.

1. Abre en Visual Studio la solución `AceriaApp.sln` de `3.3-separacion-capas`; si la primera etapa aún no la tiene, crea una solución con `dotnet new sln -n AceriaApp`.
2. Desde la raíz del punto ejecuta `dotnet sln AceriaApp.sln add src/AceriaApp.Api/AceriaApp.Api.csproj tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj` para la etapa inicial; en las siguientes etapas la solución viene heredada.
3. Verifica con `dotnet sln AceriaApp.sln list` que todas las rutas corresponden a proyectos presentes en la carpeta de esta etapa.
4. Ejecuta `dotnet build AceriaApp.sln -c Release` antes de tocar las clases y guarda su resultado como punto de comparación.

Verificación concreta. La solución abierta contiene los proyectos existentes de la etapa anterior y compila antes de los nuevos archivos.

Diagnóstico si falla.

- Abrir la solución del punto 3.2 en lugar de la del punto 3.3: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 3.2 en ejecución: el puerto queda ocupado y la compilación falla.

PASO 3.3.3 — Verificar que la carpeta `docs` contiene los documentos del punto 3.2

El punto 3.3 parte del contenido del punto 3.2, que ya incluye la carpeta `docs` con quince documentos. No se modifica ningún documento existente.

1. Abre `docs` dentro de `3.3-separacion-capas`. Si es la primera etapa y no existe, créala: botón derecho sobre la raíz del punto y `Agregar > Nueva carpeta`.
2. Comprueba que los documentos de la etapa anterior no se han modificado. A partir de aquí se añade una nueva decisión y un nuevo documento de diseño.
3. Anota los nombres de los documentos que recibes y los que crearás: `separacion-capas.md` y `ADR-012-separacion-capas.md`.
4. No reescribas un ADR anterior para explicar un diseño nuevo. Comprueba que los documentos heredados mantienen su contenido entre etapas.

Verificación concreta. la carpeta `docs` contiene los quince documentos del punto 3.2 sin modificaciones.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 3.2.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 3.3.4 — Crear el documento de separación de capas en `docs`

El documento de separación de capas es el artefacto de `docs` específico del punto 3.3. No modifica los quince documentos existentes del punto 3.2.

1. En la carpeta `docs` de `3.3-separacion-capas` crea `separacion-capas.md` mediante `Agregar > Nuevo elemento` o el editor Markdown.
2. Copia el documento de este paso y estudia sus apartados. Relaciónalo con los bloques de teoría del punto 3.3; no debe limitarse a una lista de nombres.
3. Distingue los componentes del catálogo arquitectónico de las clases y los servicios implementados en el proyecto.
4. Guarda el Markdown, abre su vista previa y anota dos decisiones observables que podrás verificar posteriormente con los tests.

Archivo que debe quedar tras este paso: `docs/separacion-capas.md`

```
# Separación entre dominio, aplicación e infraestructura — Punto 3.3

## Formas de separación

- Lógica: carpetas dentro de un mismo proyecto.
- Física: proyectos independientes.

## Proyectos del sistema

| Proyecto | Contenido | Referencias |
|----------|-----------|-------------|
| AceriaApp.Hexagonal.Dominio | Entidades y reglas de negocio | Ninguna |
| AceriaApp.Hexagonal.Aplicacion | Puertos y servicios | Dominio |
| AceriaApp.Hexagonal.Infraestructura | Repositorios y adaptadores | Aplicación y Dominio |
| AceriaApp.Api | Endpoints y configuración | Aplicación e Infraestructura |

## Regla de dependencia en la separación física

- Dominio no referencia ningún proyecto del sistema.
- Aplicación referencia solo Dominio.
- Infraestructura referencia Aplicación y Dominio.
- Api referencia Aplicación e Infraestructura.

## Ventajas

- El compilador impide las violaciones de la regla.
- Las dependencias son visibles en el archivo `.csproj`.
- Los tests referencian solo los proyectos que necesitan.
- Las bibliotecas pueden reutilizarse en distintas aplicaciones.

## Contenedor de dependencias

La presentación registra las implementaciones de los puertos. El contenedor resuelve las dependencias en
tiempo de ejecución.

## Aplicación al proyecto del curso

| Proyecto | Publicación |
|----------|-------------|
| Dominio | Biblioteca reutilizable |
| Aplicación | Biblioteca reutilizable |
| Infraestructura | Biblioteca reutilizable |
| Api | Aplicación ejecutable |

## Corrección de compatibilidad con el repositorio vigente

La implementación de partida no es `AceriaApp.Monolito.Api`, sino la Minimal API del proyecto
`M02_APIS_REST_ASPNET_CORE/PROYECTO/2.4-documentacion-openapi`. Los ejemplos de esta ficha describen
conceptos de diseño; su implementación real se incorpora al código en 3.3 y 3.4, con puertos de aplicación e
infraestructura SQL Server. No se considera registrado un adaptador solo por mencionarlo en un catálogo.
```

Verificación concreta. el archivo `docs\separacion-capas.md` existe con las siete secciones.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 3.3.5 — Crear la carpeta `SeparacionCapas` en el proyecto `AceriaApp.Api`

La carpeta `SeparacionCapas` albergará las clases de validación de la separación entre proyectos. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el Explorador de soluciones, dentro del proyecto `AceriaApp.Api`, crea la carpeta `SeparacionCapas` si no existe.
2. Comprueba que el directorio físico es `src/AceriaApp.Api/SeparacionCapas` y no está situado dentro del proyecto de tests.
3. Prepara los tres archivos de este punto: `Proyecto.cs`, `ValidadorSeparacionCapas.cs` y `CatalogoProyectos.cs`, que añadirás en los siguientes pasos.
4. No confundas estos modelos de catálogo arquitectónico con la implementación real del puerto de órdenes, que aparecerá más adelante.

Verificación concreta. la carpeta `SeparacionCapas` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 3.3.6 — Crear la clase `Proyecto`

La clase `Proyecto` representa información para analizar límites. Conviene distinguir ese objeto didáctico de los archivos `.csproj`, que son los que establecen dependencias físicas durante la compilación.

1. Crea `src/AceriaApp.Api/SeparacionCapas/Proyecto.cs` y escribe el código íntegro mostrado en el bloque asociado al paso.
2. Comprueba que el namespace es `AceriaApp.Api.SeparacionCapas`, correspondiente a la carpeta de este punto.
3. En `src/AceriaApp.Hexagonal.Dominio` crea el proyecto de clase .NET 8 y la entidad `OrdenHexagonal` del segundo bloque. No añadas EF Core al proyecto de dominio.
4. Compila el proyecto de dominio por separado y comprueba que no referencia ASP.NET Core, aplicación ni infraestructura.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/SeparacionCapas/Proyecto.cs`

```
namespace AceriaApp.Api.SeparacionCapas;

public class Proyecto
{
    public string Nombre { get; init; } = string.Empty;
    public string Capa { get; init; } = string.Empty;
    public IReadOnlyList<string> Referencias { get; init; } = Array.Empty<string>();
}
```

Archivo que debe quedar tras este paso: `src/AceriaApp.Hexagonal.Dominio/AceriaApp.Hexagonal.Dominio.csproj`

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
</Project>
```

Archivo que debe quedar tras este paso: `src/AceriaApp.Hexagonal.Dominio/Entidades/OrdenHexagonal.cs`

```
namespace AceriaApp.Hexagonal.Dominio;

/// <summary>Entidad del núcleo: no depende de HTTP, EF Core ni SQL Server.</summary>
public sealed class OrdenHexagonal
{
    private OrdenHexagonal() { } // Materialización por EF Core.
    public OrdenHexagonal(string cliente, string material, int cantidad)
    {
        if (string.IsNullOrWhiteSpace(cliente) || string.IsNullOrWhiteSpace(material))
            throw new ArgumentException("Cliente y material son obligatorios.");
        if (cantidad <= 0) throw new ArgumentOutOfRangeException(nameof(cantidad));
        Id = Guid.NewGuid();
        Cliente = cliente.Trim(); Material = material.Trim(); Cantidad = cantidad;
        FechaCreacion = DateTime.UtcNow;
    }
    public Guid Id { get; private set; }
    public string Cliente { get; private set; } = string.Empty;
    public string Material { get; private set; } = string.Empty;
    public int Cantidad { get; private set; }
    public string Estado { get; private set; } = "Pendiente";
    public DateTime FechaCreacion { get; private set; }
    public void Finalizar()
    {
        if (Estado == "Finalizada") throw new InvalidOperationException("La orden ya estaba finalizada.");
        Estado = "Finalizada";
    }
}
```

Verificación concreta. la clase `Proyecto` compila sin errores y expone tres propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar `Nombre` y `Capa` con `string.Empty`: el compilador emite advertencias.
- No inicializar `Referencias` con `Array.Empty<string>()`: el compilador emite advertencia.

PASO 3.3.7 — Crear la clase `ValidadorSeparacionCapas`

`ValidadorSeparacionCapas` contrasta relaciones declaradas. La revisión real de las referencias de proyectos debe acompañar al resultado de sus pruebas para no aceptar fronteras solo por su denominación.

1. Abre o crea `src/AceriaApp.Api/SeparacionCapas/ValidadorSeparacionCapas.cs` según el bloque de C# de este paso.
2. Escribe la clase pública `ValidadorSeparacionCapas` y el record de resultado; comprueba sus tipos de entrada y las ramas de error.
3. Crea `AceriaApp.Hexagonal.Aplicacion` referenciando solo Dominio; añade `IOrdenRepositorio` como puerto y `OrdenesCasoUso` sin dependencias de SQL Server.
4. Compila Aplicación y verifica que `IOrdenRepositorio` ofrece operaciones asíncronas mientras las reglas de creación permanecen en los casos de uso.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/SeparacionCapas/ValidadorSeparacionCapas.cs`

```
namespace AceriaApp.Api.SeparacionCapas;
```

```
public class ValidadorSeparacionCapas
{
    private static readonly Dictionary<string, string[]> ReferenciasPermitidas = new()
    {
        { "Dominio", Array.Empty<string>() },
        { "Aplicacion", new[] { "Dominio" } },
        { "Infraestructura", new[] { "Aplicacion", "Dominio" } },
        { "Api", new[] { "Aplicacion", "Infraestructura" } }
    };

    public ResultadoSeparacion Validar(IEnumerable<Proyecto> proyectos)
    {
        var errores = new List<string>();

        foreach (var proyecto in proyectos)
        {
            if (!ReferenciasPermitidas.ContainsKey(proyecto.Capa))
            {
                errores.Add($"Capa desconocida: '{proyecto.Capa}'.");
                continue;
            }

            var permitidas = ReferenciasPermitidas[proyecto.Capa];
            foreach (var referencia in proyecto.Referencias)
            {
                if (!permitidas.Contains(referencia))
                {
                    errores.Add($"El proyecto '{proyecto.Nombre}' de la capa '{proyecto.Capa}' no puede
referenciar '{referencia}'.");
                }
            }
        }

        return new ResultadoSeparacion(errores.Count == 0, errores);
    }
}

public record ResultadoSeparacion(bool EsValida, IReadOnlyList<string> Errores);
```

Archivo que debe quedar tras este paso: `src/AceriaApp.Hexagonal.Aplicacion/AceriaApp.Hexagonal.Aplicacion.csproj`

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup><ProjectReference Include="../AceriaApp.Hexagonal.Dominio/AceriaApp.Hexagonal.Dominio.csproj" />
></ItemGroup>
</Project>
```

Archivo que debe quedar tras este paso: `src/AceriaApp.Hexagonal.Aplicacion/Puertos/IOrdenRepositorio.cs`

```
using AceriaApp.Hexagonal.Dominio;

namespace AceriaApp.Hexagonal.Aplicacion.Puertos;

/// <summary>Puerto de salida. Aplicación no conoce EF Core ni el proveedor SQL.</summary>
public interface IOrdenRepositorio
{
    Task<IReadOnlyList<OrdenHexagonal>> ListarAsync(CancellationToken ct = default);
    Task<OrdenHexagonal?> ObtenerAsync(Guid id, CancellationToken ct = default);
    Task AgregarAsync(OrdenHexagonal orden, CancellationToken ct = default);
    Task GuardarAsync(OrdenHexagonal orden, CancellationToken ct = default);
}
```

Archivo que debe quedar tras este paso: `src/AceriaApp.Hexagonal.Aplicacion/CasosUso/OrdenesCasoUso.cs`

```
using AceriaApp.Hexagonal.Aplicacion.Puertos;
using AceriaApp.Hexagonal.Dominio;
```

```
namespace AceriaApp.Hexagonal.Aplicacion.CasosUso;

/// <summary>Casos de uso, independientes de HTTP y del adaptador de persistencia.</summary>
public sealed class OrdenesCasoUso(IOrdenRepositorio repositorio)
{
    public Task<IReadOnlyList<OrdenHexagonal>> ListarAsync(CancellationToken ct = default) =>
        repositorio.ListarAsync(ct);

    public Task<OrdenHexagonal?> ObtenerAsync(Guid id, CancellationToken ct = default) =>
        repositorio.ObtenerAsync(id, ct);

    public async Task<OrdenHexagonal> CrearAsync(string cliente, string material, int cantidad,
CancellationToken ct = default)
    {
        if (cantidad > 500) throw new ArgumentOutOfRangeException(nameof(cantidad), "Capacidad máxima:
500.");
        var orden = new OrdenHexagonal(cliente, material, cantidad);
        await repositorio.AgregarAsync(orden, ct);
        return orden;
    }

    public async Task<OrdenHexagonal?> FinalizarAsync(Guid id, CancellationToken ct = default)
    {
        var orden = await repositorio.ObtenerAsync(id, ct);
        if (orden is null) return null;
        orden.Finalizar();
        await repositorio.GuardarAsync(orden, ct);
        return orden;
    }
}
```

Verificación concreta. la clase `ValidadorSeparacionCapas` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `Dictionary` y `List<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` no se resuelve.
- Olvidar el `using System;`: `Array.Empty` no se resuelve.

PASO 3.3.8 — Añadir los `using` necesarios en `ValidadorSeparacionCapas`

Los tipos `Dictionary` y `List<string>` pertenecen a `System.Collections.Generic`. El método `Contains` pertenece a `System.Linq`. El método `Array.Empty` pertenece a `System`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. Abre `src/AceriaApp.Api/SeparacionCapas/ValidadorSeparacionCapas.cs` y localiza las directivas `using` de la parte superior.
2. Verifica que `System.Collections.Generic`, `System.Linq` y `System` están disponibles según las colecciones o métodos usados; con ImplicitUsings activado algunos llegan automáticamente.
3. Añade solo el import que realmente falte. No copies una segunda clase ni sustituyas la implementación del validador por un comentario.
4. Guarda y compila. Deja registrado qué miembro concreto requería el namespace, como `List<string>`, `Where` o `Contains`.

Verificación concreta. `ValidadorSeparacionCapas.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo uno o dos de los tres usings: los tipos del otro no se resuelven.
- Añadir los usings después del namespace: el compilador no los reconoce.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 3.3.9 — Crear la clase `CatalogoProyectos`

`CatalogoProyectos` agrupa propuestas de separación y las clasifica, pero no modifica el ensamblado que puede referenciar a otro. La comprobación posterior debe recorrer también la solución .NET.

1. Crea `src/AceriaApp.Api/SeparacionCapas/CatalogoProyectos.cs` con el catálogo y los valores de ejemplo indicados en el código del laboratorio.
2. Identifica las reglas o registros que el catálogo declara y cuáles pertenecen únicamente al ejercicio conceptual; no los des por registrados en DI.
3. Crea `AceriaApp.Hexagonal.Infraestructura` y sus dependencias EF Core SQL Server; implementa `HexagonalDbContext`, `OrdenRepositorioSqlServer` y el registro de servicios.
4. Compila Infraestructura y comprueba que implementa `IOrdenRepositorio`, mientras Aplicación no importa namespaces EF Core.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/SeparacionCapas/CatalogoProyectos.cs`

```
namespace AceriaApp.Api.SeparacionCapas;

public class CatalogoProyectos
{
    public static List<Proyecto> ProyectosPorDefecto()
    {
        return new List<Proyecto>
        {
            new()
            {
                Nombre = "AceriaApp.Hexagonal.Dominio",
                Capa = "Dominio",
                Referencias = Array.Empty<string>()
            },
            new()
            {
                Nombre = "AceriaApp.Hexagonal.Aplicacion",
                Capa = "Aplicacion",
                Referencias = new[] { "Dominio" }
            },
            new()
            {
                Nombre = "AceriaApp.Hexagonal.Infraestructura",
                Capa = "Infraestructura",
                Referencias = new[] { "Aplicacion", "Dominio" }
            },
            new()
            {
                Nombre = "AceriaApp.Api",
                Capa = "Api",
                Referencias = new[] { "Aplicacion", "Infraestructura" }
            }
        };
    }
}
```

Archivo que debe quedar tras este paso: `src/AceriaApp.Hexagonal.Infraestructura/AceriaApp.Hexagonal.Infraestructura.csproj`

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</ImplicitUsings></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../AceriaApp.Hexagonal.Dominio/AceriaApp.Hexagonal.Dominio.csproj" />
    <ProjectReference Include="../AceriaApp.Hexagonal.Aplicacion/AceriaApp.Hexagonal.Aplicacion.csproj" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.21" />
  </ItemGroup>
</Project>
```

Archivo que debe quedar tras este paso: `src/AceriaApp.Hexagonal.Infraestructura/Datos/HexagonalDbContext.cs`

```
using AceriaApp.Hexagonal.Dominio;
using Microsoft.EntityFrameworkCore;

namespace AceriaApp.Hexagonal.Infraestructura.Datos;

public sealed class HexagonalDbContext(DbContextOptions<HexagonalDbContext> options) : DbContext(options)
{
    public DbSet<OrdenHexagonal> Ordenes => Set<OrdenHexagonal>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        var e = modelBuilder.Entity<OrdenHexagonal>();
        e.ToTable("OrdenesHexagonales");
        e.HasKey(o => o.Id);
        e.Property(o => o.Cliente).HasMaxLength(100).IsRequired();
        e.Property(o => o.Material).HasMaxLength(100).IsRequired();
        e.Property(o => o.Estado).HasMaxLength(24).IsRequired();
    }
}
```

Archivo que debe quedar tras este paso: `src/AceriaApp.Hexagonal.Infraestructura/Adaptadores/OrdenRepositorioSqlServer.cs`

```
using AceriaApp.Hexagonal.Aplicacion.Puertos;
using AceriaApp.Hexagonal.Dominio;
using AceriaApp.Hexagonal.Infraestructura.Datos;
using Microsoft.EntityFrameworkCore;

namespace AceriaApp.Hexagonal.Infraestructura.Adaptadores;

public sealed class OrdenRepositorioSqlServer(HexagonalDbContext db) : IOrdenRepositorio
{
    public async Task<IReadOnlyList<OrdenHexagonal>> ListarAsync(CancellationToken ct = default) =>
        await db.Ordenes.AsNoTracking().OrderBy(o => o.FechaCreacion).ToListAsync(ct);

    public Task<OrdenHexagonal?> ObtenerAsync(Guid id, CancellationToken ct = default) =>
        db.Ordenes.FirstOrDefaultAsync(o => o.Id == id, ct);

    public async Task AgregarAsync(OrdenHexagonal orden, CancellationToken ct = default)
    {
        await db.Ordenes.AddAsync(orden, ct);
        await db.SaveChangesAsync(ct);
    }

    public async Task GuardarAsync(OrdenHexagonal orden, CancellationToken ct = default)
    {
        db.Ordenes.Update(orden);
        await db.SaveChangesAsync(ct);
    }
}
```

Archivo que debe quedar tras este paso: `src/AceriaApp.Hexagonal.Infraestructura/ServiciosHexagonales.cs`

```
using AceriaApp.Hexagonal.Aplicacion.CasosUso;
using AceriaApp.Hexagonal.Aplicacion.Puertos;
using AceriaApp.Hexagonal.Infraestructura.Adaptadores;
using AceriaApp.Hexagonal.Infraestructura.Datos;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

namespace AceriaApp.Hexagonal.Infraestructura;

public static class ServiciosHexagonales
{
    public static IServiceCollection AddServiciosHexagonales(this IServiceCollection services, IConfiguration
configuration)
    {
        services.AddDbContext<HexagonalDbContext>(options =>
            options.UseSqlServer(configuration.GetConnectionString("AceriaHexagonal")));
```

```
        services.AddScoped<IOrdenRepositorio, OrdenRepositorioSqlServer>();
        services.AddScoped<OrdenesCasoUso>();
        return services;
    }
}
```

Verificación concreta. la clase `CatalogoProyectos` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System;`: `Array.Empty` no se resuelve.
- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades: el compilador emite advertencias.

PASO 3.3.10 — Añadir los `using` necesarios en `CatalogoProyectos`

El tipo `List<T>` pertenece a `System.Collections.Generic` y `Array.Empty` pertenece a `System`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. Abre `src/AceriaApp.Api/SeparacionCapas/CatalogoProyectos.cs` y observa las colecciones e importaciones utilizadas por el catálogo.
2. Revisa `using System;`, `using System.Linq;` y `using System.Collections.Generic;` cuando corresponda; conserva solo lo que la compilación necesite.
3. No añadas dependencias tecnológicas nuevas al dominio para resolver tipos básicos ni vuelvas a crear el archivo de catálogo.
4. Compila y confirma que los métodos del catálogo pueden llamarse desde el proyecto de tests sin errores de namespace.

Verificación concreta. `CatalogoProyectos.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System`: `Array.Empty` no se resuelve.
- Añadir los usings después del namespace: el compilador no los reconoce.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 3.3.11 — Crear el proyecto de tests del punto 3.3

El punto 3.3 añade un proyecto de tests específico para las clases de separación de capas. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Dentro de `tests` crea el proyecto xUnit `AceriaApp.SeparacionCapas.Tests` con destino `net8.0`. No modifiques ni sustituyas `AceriaApp.Api.Tests`, que contiene las pruebas HTTP heredadas de M02.
2. Crea el archivo `tests/AceriaApp.SeparacionCapas.Tests/AceriaApp.SeparacionCapas.Tests.csproj` y escribe el XML completo del bloque del laboratorio, con xUnit y FluentAssertions 6.12.2.
3. Comprueba que incluye un `ProjectReference` hacia `../../src/AceriaApp.Api/AceriaApp.Api.csproj`; agrega el nuevo proyecto con `dotnet sln AceriaApp.sln add tests/AceriaApp.SeparacionCapas.Tests/AceriaApp.SeparacionCapas.Tests.csproj`.
4. Ejecuta `dotnet restore AceriaApp.sln` y comprueba que no se ha generado un test `UnitTest1` no previsto.

Archivo que debe quedar tras este paso: `tests/AceriaApp.SeparacionCapas.Tests/AceriaApp.SeparacionCapas.Tests.csproj`

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsTestProject>true</IsTestProject><IsPackable>false</IsPackable></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
```

```
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El proyecto de tests `AceriaApp.SeparacionCapas.Tests` restaura, referencia a `AceriaApp.Api` y no sustituye ninguna suite heredada.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 3.2: viola la regla de no modificar archivos del punto anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 3.3.12 — Añadir la referencia del proyecto de tests a la API `AceriaApp.Api`

La referencia permite que el proyecto de tests acceda a las clases de `SeparacionCapas` del proyecto `AceriaApp.Api`.

1. En `src/AceriaApp.Api/AceriaApp.Api.csproj`, localiza el cierre `</Project>` y añade las referencias a Aplicación e Infraestructura del fragmento XML.
2. Agrega los tres proyectos de clase a `AceriaApp.sln` mediante `dotnet sln add`, junto con el proyecto de tests.
3. Desde la raíz del punto ejecuta `dotnet build AceriaApp.sln` y revisa que el proyecto de Dominio no referencia ningún proyecto exterior.
4. La API compila con esas referencias, aunque los endpoints nuevos se conectarán en 3.4 y los endpoints de M02 permanecen operativos.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/AceriaApp.Api.csproj` (modificación; buscar `</Project>`)

```
  <ItemGroup>
    <ProjectReference Include="../AceriaApp.Hexagonal.Aplicacion/AceriaApp.Hexagonal.Aplicacion.csproj" />
    <ProjectReference Include="../AceriaApp.Hexagonal.Infraestructura/
AceriaApp.Hexagonal.Infraestructura.csproj" />
  </ItemGroup>
```

Verificación concreta. El proyecto de tests `AceriaApp.SeparacionCapas.Tests` restaura, referencia a `AceriaApp.Api` y no sustituye ninguna suite heredada.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 3.3.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Abre el `.csproj` de `AceriaApp.SeparacionCapas.Tests` y localiza el `PackageReference` a `FluentAssertions` versión 6.12.2.
2. Ejecuta `dotnet list tests/AceriaApp.SeparacionCapas.Tests/AceriaApp.SeparacionCapas.Tests.csproj package` y comprueba que el paquete consta en el proyecto correcto.
3. Relaciona el método `Should()` de los ejemplos de tests con FluentAssertions, y `[Fact]` con xUnit.

4. No instales una segunda versión del mismo paquete: basta con restaurar NuGet desde la solución.

Verificación concreta. El proyecto de tests `AceriaApp.SeparacionCapas.Tests` restaura, referencia a `AceriaApp.Api` y no sustituye ninguna suite heredada.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 3.3.14 — Crear el archivo `ValidadorSeparacionCapasTests.cs`

Las pruebas del validador distinguen relaciones permitidas y prohibidas. Para asegurar una separación real, compara esas reglas con los ProjectReference que admite el compilador.

1. Crea `tests/AceriaApp.SeparacionCapas.Tests/ValidadorSeparacionCapasTests.cs` y escribe completo el primer bloque de pruebas de este punto.
2. Revisa el `using AceriaApp.Api.SeparacionCapas` para que coincida con el namespace actualizado y conserva `using Xunit;` y `using FluentAssertions;`.
3. Localiza los casos correctos e incorrectos del validador y comprueba que las aserciones describen resultados de código ejecutable.
4. Guarda, compila y ejecuta el test específico. No cambies las expectativas para ocultar una regla equivocada.

Archivo que debe quedar tras este paso: `tests/AceriaApp.SeparacionCapas.Tests/ValidadorSeparacionCapasTests.cs`

```
using AceriaApp.Api.SeparacionCapas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.SeparacionCapas.Tests;

public class ValidadorSeparacionCapasTests
{
    private readonly ValidadorSeparacionCapas _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoProyectos.ProyectosPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_DominioConReferencia_DevuelveError()
    {
        var proyectos = new List<Proyecto>
        {
            new()
            {
                Nombre = "AceriaApp.Hexagonal.Dominio",
                Capa = "Dominio",
                Referencias = new[] { "Infraestructura" }
            }
        };

        var resultado = _validador.Validar(proyectos);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no puede referenciar"));
    }
```

```
    [Fact]
    public void Validar_AplicacionReferenciandoInfraestructura_DevuelveError()
    {
        var proyectos = new List<Proyecto>
        {
            new()
            {
                Nombre = "AceriaApp.Hexagonal.Aplicacion",
                Capa = "Aplicacion",
                Referencias = new[] { "Infraestructura" }
            }
        };

        var resultado = _validador.Validar(proyectos);

        resultado.EsValida.Should().BeFalse();
    }

    [Fact]
    public void Validar_CapaDesconocida_DevuelveError()
    {
        var proyectos = new List<Proyecto>
        {
            new() { Nombre = "ProyectoX", Capa = "Inventada" }
        };

        var resultado = _validador.Validar(proyectos);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Capa desconocida"));
    }
}
```

Verificación concreta. Los tests de `AceriaApp.SeparacionCapas.Tests` compilan y ejecutan las aserciones propuestas para `SeparacionCapas`.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.SeparacionCapas;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de capa desconocida: el mensaje de error no se verifica.

PASO 3.3.15 — Crear el archivo `CatalogoProyectosTests.cs`

Los tests verifican que el catálogo por defecto contiene los proyectos esperados y que las referencias son las correctas.

1. Crea `tests/AceriaApp.SeparacionCapas.Tests/CatalogoProyectosTests.cs` y escribe el segundo bloque de pruebas del catálogo.
2. Comprueba que las llamadas a los métodos de ejemplo de `{catalog}` son compatibles con las propiedades que realmente expone el código.
3. Guarda y ejecuta `dotnet test tests/AceriaApp.SeparacionCapas.Tests/AceriaApp.SeparacionCapas.Tests.csproj -c Release`, de forma que se ejecuten todas las pruebas nuevas.
4. Verifica que las aserciones prueban los valores declarados, pero no confunde un catálogo didáctico con un registro en tiempo de ejecución.

Archivo que debe quedar tras este paso: `tests/AceriaApp.SeparacionCapas.Tests/CatalogoProyectosTests.cs`

```
using AceriaApp.Api.SeparacionCapas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.SeparacionCapas.Tests;
```

```
public class CatalogoProyectosTests
{
    [Fact]
    public void ProyectosPorDefecto_ContieneCuatroProyectos()
    {
        var proyectos = CatalogoProyectos.ProyectosPorDefecto();

        proyectos.Should().HaveCount(4);
    }

    [Fact]
    public void ProyectosPorDefecto_DominioSinReferencias()
    {
        var proyectos = CatalogoProyectos.ProyectosPorDefecto();

        var dominio = proyectos.First(p => p.Capa == "Dominio");
        dominio.Referencias.Should().BeEmpty();
    }

    [Fact]
    public void ProyectosPorDefecto_AplicacionSoloReferenciaDominio()
    {
        var proyectos = CatalogoProyectos.ProyectosPorDefecto();

        var aplicacion = proyectos.First(p => p.Capa == "Aplicacion");
        aplicacion.Referencias.Should().HaveCount(1);
        aplicacion.Referencias.Should().Contain("Dominio");
    }
}
```

Verificación concreta. Los tests de `AceriaApp.SeparacionCapas.Tests` compilan y ejecutan las aserciones propuestas para `SeparacionCapas`.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.SeparacionCapas;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `First` no se resuelve.
- No verificar las referencias del dominio: la capa más interna no se comprueba.

PASO 3.3.16 — Compilar la solución completa

Compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

1. Abre la terminal junto a `AceriaApp.sln` y ejecuta `dotnet build AceriaApp.sln -c Release` después de introducir todos los archivos del punto.
2. Comprueba que compilan la API de M02, el proyecto nuevo de tests y todos los proyectos de etapas anteriores.
3. Si aparece un error, identifica el primer archivo y la primera línea antes de tocar otra clase. Revisa referencias y namespaces.
4. Registra la salida de la compilación y no avances hasta obtener cero errores.

Verificación concreta. La solución completa compila en Release con cero errores. La capa de dominio no referencia EF Core ni ASP.NET Core.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 3.3.17 — Ejecutar todos los tests del punto 3.3

Ejecutar los tests del punto 3.3 verifica que las clases de separación de capas funcionan correctamente.

1. Ejecuta `dotnet test tests/AceriaApp.SeparacionCapas.Tests/AceriaApp.SeparacionCapas.Tests.csproj -c Release` desde la raíz del punto.
2. Comprueba que los métodos `[Fact]` de `ValidadorSeparacionCapasTests` y `CatalogoProyectosTests` pasan; el resumen numérico exacto aparece en la tabla de resultados de este módulo.
3. Ejecuta un caso inválido del validador en el depurador y relaciona el mensaje de error con la línea que lo produce.
4. Conserva la salida de tests como evidencia y confirma que los resultados no dependen de SQL Server para evaluar reglas puras.

Verificación concreta. La suite nueva de 3.3 supera 7 pruebas de catálogo; en 3.4 se añaden además 4 pruebas HTTP del adaptador real.

Diagnóstico si falla.

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 3.3.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 3.3 pasan conjuntamente.

1. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después de compilar toda la solución en Release.
2. Comprueba que pasan los tests de M02 y todos los nuevos del módulo M03 acumulados hasta este punto.
3. Si falla alguna suite previa, vuelve a comprobar si el último cambio alteró una dependencia o una configuración del servidor.
4. Registra las pruebas totales, fallidas y omitidas y compáralas con el resultado previsto para esta etapa.

Verificación concreta. Pruebas acumuladas previstas para 3.3: 38, 0 fallidas. Comprueba estos valores en la salida de `dotnet test`.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

PASO 3.3.19 — Crear el README del punto 3.3

El README del punto 3.3 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 3.2.

1. Abre `README.md` en la raíz del punto `3.3-separacion-capas` y sustitúyelo por el README específico del bloque de este paso.
2. Comprueba que los nombres de proyectos y las pruebas acumuladas coinciden con la solución y con la salida de `dotnet test` de esta etapa.

3. Revisa la ubicación del documento `docs` y los comandos `dotnet build` y `dotnet test` que utilizará el alumno.
4. Guarda la versión del punto actual sin sobrescribir el README de la etapa anterior.

Archivo que debe quedar tras este paso: `README.md`

```
# Punto 3.3 — Carpeta del punto 3.3 a partir del punto 3.2

**Curso:** Microservicios y arquitectura hexagonal con ASP.NET Core 8
**Código de referencia:** AceriaApp, en continuidad con M02 2.4.

## Objetivo

el documento de separación de capas es el artefacto de `docs` específico del punto 3.3. No modifica los
quince documentos existentes del punto 3.2.

## Estado verificado del punto

- 8 proyectos en la solución `AceriaApp.sln` al finalizar este punto.
- 38 pruebas xUnit acumuladas previstas (incluyendo las 16 de M02 y las nuevas del módulo 3).
- Clases del catálogo `SeparacionCapas` en `src/AceriaApp.Api/SeparacionCapas`.
- Documentación `docs/separacion-capas.md` y decisión `docs/ADR-012-separacion-capas.md`.
- Los endpoints REST del módulo 2 se conservan sin cambios funcionales.

## Cómo ejecutarlo

1. Abrir el archivo `AceriaApp.sln` de esta carpeta en Visual Studio, con SDK .NET 8.
2. Ejecutar `dotnet restore AceriaApp.sln` y después `dotnet build AceriaApp.sln -c Release`.
3. Ejecutar `dotnet test AceriaApp.sln -c Release --no-build`. La suite M03 se construye por adición desde
M02.
4. La aplicación principal utiliza SQL Server Express LocalDB; los tests HTTP mantienen SQL Server y crean
bases exclusivas independientes mediante `BaseSqlServer.Crear()`.
5. En 3.4, los endpoints hexagonales reales están en `/api/v1/hexagonal/ordenes` y utilizan un repositorio EF
Core a través de un puerto de aplicación.

## Qué se ha añadido

la clase `Proyecto` modela un proyecto con su nombre, capa y lista de referencias. Es la unidad de
información del validador de separación.
```

Verificación concreta. `README.md` describe correctamente los 8 proyectos y 38 pruebas acumuladas del curso vigente.

Diagnóstico si falla.

- Modificar el README del punto 3.2: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 3.2: rompe la separación.

PASO 3.3.20 — Crear el ADR-012 del punto 3.3

El ADR-012 documenta la decisión de adoptar la separación física. Es un artefacto nuevo del punto 3.3 que no modifica los ADR existentes.

1. En la carpeta `docs` crea `ADR-012-separacion-capas.md` y copia el documento de decisión del bloque del laboratorio.
2. Lee Estado, Contexto, Decisión, Alternativas y Consecuencias y relaciona el ADR con el código nuevo de la etapa.
3. Explica por qué no basta con declarar nombres de capas si el código puede saltarse la interfaz o registrar una dependencia incorrecta.
4. Comprueba que los ADR anteriores están presentes y no han sido sobrescritos.

Archivo que debe quedar tras este paso: `docs/ADR-012-separacion-capas.md`

```
# ADR-012: Separación física entre dominio, aplicación e infraestructura

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado la arquitectura hexagonal con puertos y adaptadores. Es necesario decidir
si la separación entre capas se realiza de forma lógica o física.

## Decisión
Se adopta la separación física: cada capa se agrupa en un proyecto independiente. El proyecto de dominio no
referencia ningún otro proyecto. El proyecto de aplicación referencia el dominio. El proyecto de
infraestructura referencia la aplicación y el dominio. El proyecto de presentación referencia la aplicación y
la infraestructura. La separación se valida con un test que comprueba las referencias.

## Alternativas consideradas
- Separación lógica en carpetas: descartado por permitir violaciones invisibles.
- Un único proyecto para todas las capas: descartado por romper la regla de dependencia.

## Consecuencias
- Positivas: el compilador impide las violaciones, las dependencias son visibles, los tests referencian solo
lo necesario.
- Negativas: la configuración de referencias requiere disciplina.
- Mitigación: validación automática con el validador.

## Aplicación en AceriaApp

Se parte de la API real y probada de M02, se conservan sus endpoints y se añaden reglas de dependencia
verificadas en código. Los proyectos de Dominio, Aplicación e Infraestructura deben obedecer la dirección de
referencias; en 3.4 se expone un nuevo adaptador HTTP funcional. El catálogo describe las relaciones
arquitectónicas; cada implementación se comprueba en su proyecto correspondiente.
```

Verificación concreta. el archivo `docs\ADR-012-separacion-capas.md` existe con las cinco secciones.

Diagnóstico si falla.

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-012`: rompe la convención.

PASO 3.3.21 — Verificar que no se ha modificado ningún archivo del punto 3.2

La comparación con `fc` verifica que todos los documentos del punto 3.2 no se han modificado.

1. Localiza el mismo documento heredado tanto en `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.2-puertos-adaptadores` como en `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.3-separacion-capas`.
2. Compara sus hashes SHA-256 o usa un diff y confirma que no se reescribió al introducir el nuevo punto.
3. Distingue de él los archivos añadidos en este punto: catálogo, validador, pruebas y documentación nueva.
4. Documenta cualquier diferencia inesperada; el nuevo código no exige alterar los archivos de etapas anteriores.

Verificación concreta. Los archivos heredados de `docs` y los endpoints M02 no han sido sobrescritos.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Olvidar el flag `/L`: `fc` compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 3.3.22 — Verificar la estructura del punto 3.3

La verificación estructural confirma que el punto 3.3 añade archivos nuevos sin modificar los existentes.

1. Desde la raíz de `M03_ARQUITECTURA_HEXAGONAL/PROYECTO` ejecuta `tree /F 3.3-separacion-capas` en Windows.
2. Localiza `src/AceriaApp.Api/SeparacionCapas`, `tests/AceriaApp.SeparacionCapas.Tests`, `README.md` y `docs/separacion-capas.md`.
3. Comprueba los proyectos adicionales de dominio, aplicación e infraestructura si ya fueron introducidos por el punto 3.3.
4. Distingue los archivos del proyecto de los temporales `obj/` y `bin/`; ambos deben quedar excluidos de la entrega.

Verificación concreta. Los archivos que aparecen en el árbol corresponden a los fragmentos creados hasta este paso, y no hay código pendiente en carpetas fuera de la solución.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 3.2: viola la regla de no modificar.
- Omitir la carpeta `SeparacionCapas`: los tests no encuentran las clases.

PASO 3.3.23 — Verificar el `.gitignore` del punto 3.3

El `.gitignore` se hereda del punto 3.2 sin modificaciones. La verificación confirma que no se ha tocado.

1. Abre el `.gitignore` de la etapa; si falta, crea uno que excluya `.vs/`, `bin/`, `obj/`, archivos personales y resultados de prueba.
2. Ejecuta `git status --short` en tu copia de trabajo y comprueba que no aparecen archivos binarios ni credenciales.
3. No ignores `src`, `tests`, `docs` ni las soluciones `.sln`, pues son parte del entregable reproducible.
4. Conserva el `.gitignore` heredado sin borrarlo ni modificar el de etapas anteriores.

Verificación concreta. No se incluyen `bin`, `obj`, contraseñas ni resultados de pruebas en el estado Git.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 3.2: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 3.3.24 — Hacer el commit del punto 3.3 y verificar el estado final

El commit del punto 3.3 registra el estado acumulado del proyecto. Los commits de cada punto permiten comparar el estado del proyecto entre puntos consecutivos y entre módulos completos.

1. Antes de terminar 3.3, vuelve a ejecutar `dotnet build AceriaApp.sln -c Release` y `dotnet test AceriaApp.sln -c Release --no-build`.
2. Lee el reto de ampliación incluido tras los pasos obligatorios; si lo realizas, utiliza una copia aparte y deja los tests base intactos.
3. Si tu copia está bajo Git, ejecuta `git status`, añade solo lo trabajado y crea un commit descriptivo, sin modificar la rama canónica del profesor.
4. Conserva el resumen de pruebas y compara el proyecto final con la carpeta de referencia del punto; ambas deben representar el mismo código.

Verificación concreta. El punto termina con compilación y suite de pruebas completa, sin modificar la referencia de la etapa anterior.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Esta ampliación añade una operación de resumen al validador y una prueba de su resultado. **Realízala después de superar la etapa básica y sus tests.** Incorpora cada fragmento en la clase indicada y conserva el namespace del proyecto en el que trabajas.

Reto: añadir un método `Resumen` a `ValidadorSeparacionCapas` que devuelva el número de proyectos y el número total de referencias, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorSeparacionCapas.cs`, añadir el siguiente `record` al final del archivo:

```
public record ResumenSeparacion(int Proyectos, int Referencias);
```

Paso 2: Explicar línea por línea:

- **Línea 1:** `public record ResumenSeparacion(` → declara un contrato inmutable.
- **Línea 2:** `int Proyectos,` → número de proyectos del sistema.
- **Línea 3:** `int Referencias);` → número total de referencias.

Paso 3: En `ValidadorSeparacionCapas.cs`, añadir el siguiente método dentro de la clase:

```
public ResumenSeparacion Resumen(IEnumerable<Proyecto> proyectos)
{
    var lista = proyectos.ToList();
    var referencias = lista.Sum(p => p.Referencias.Count);
    return new ResumenSeparacion(lista.Count, referencias);
}
```

Paso 4: Explicar línea por línea:

- **Línea 1:** `public ResumenSeparacion Resumen(IEnumerable<Proyecto> proyectos)` → declara el método.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `var lista = proyectos.ToList();` → materializa la lista.
- **Línea 4:** `var referencias = lista.Sum(p => p.Referencias.Count);` → suma las referencias.
- **Línea 5:** `return new ResumenSeparacion(lista.Count, referencias);` → devuelve el resumen.
- **Línea 6:** `}` → cierra el cuerpo.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorSeparacionCapasTests.cs`, añadir el siguiente test:

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CuentaCorrectamente()
{
    var resumen = _validador.Resumen(CatalogoProyectos.ProyectosPorDefecto());

    resumen.Proyectos.Should().Be(4);
    resumen.Referencias.Should().Be(5);
}
```

Paso 7: Explicar línea por línea:

- **Línea 1:** `[Fact]` → marca el método como test de xUnit.
- **Línea 2:** `public void Resumen_ConCatalogoPorDefecto_CuentaCorrectamente()` → declara el test.
- **Línea 3:** `{` → abre el cuerpo.
- **Línea 4:** `var resumen = _validador.Resumen(CatalogoProyectos.ProyectosPorDefecto());` → invoca el método.
- **Línea 5:** `resumen.Proyectos.Should().Be(4);` → verifica el número de proyectos.
- **Línea 6:** `resumen.Referencias.Should().Be(5);` → verifica las cinco referencias: una de Aplicación, dos de Infraestructura y dos de la API.
- **Línea 7:** `}` → cierra el cuerpo.

Paso 8: Ejecutar `dotnet test tests\AceriaApp.SeparacionCapas.Tests\AceriaApp.SeparacionCapas.Tests.csproj` y verificar que pasan los ocho tests.

Paso 9: Verificar que el método `Resumen` no modifica ningún archivo del punto 3.2.

Paso 10: Hacer commit con `git commit -m "Reto 3.3: método Resumen en ValidadorSeparacionCapas"`.

Verificación: la salida de `dotnet test` muestra `Passed: 8`.

Resultado verificable y cierre del punto 3.3

Código final de referencia: `PROYECTO/3.3-separacion-capas/`. El contenido que debe crear el alumno está especificado por los bloques con su ruta exacta y el tipo de modificación. Comprueba el resultado compilando y ejecutando las pruebas de la etapa.

Continuidad: las dependencias existentes y la documentación heredada se conservan; el avance del punto 3.3 prepara el siguiente sin reescribir la etapa anterior.

Punto 3.4 — Implementación en ASP.NET Core

Ejecutar la arquitectura hexagonal desde HTTP hasta SQL Server

En 3.4 la API de ASP.NET Core debe actuar como adaptador de entrada: recibe una petición, invoca la aplicación y deja que infraestructura resuelva el acceso a SQL Server a través de los puertos establecidos. El registro de dependencias debe componer estas piezas sin trasladar EF Core al dominio.

Continuaremos desde la solución de 3.3 y verificaremos tanto las reglas del catálogo de registro como las conexiones reales de la aplicación. Compilar y pasar tests de clasificación resulta necesario, pero la evidencia de una integración hexagonal funcional exige también peticiones HTTP y comprobaciones persistentes en SQL Server.

Objetivos del punto. el documento de implementación en ASP.NET Core es el artefacto de `docs` específico del punto 3.4. Incluye el cierre del módulo 3 con el resumen del estado del proyecto. No modifica los dieciséis documentos existentes del punto 3.3.

Carpeta de trabajo: `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.4-implementacion-aspnet-core`. Esta etapa continúa la anterior sin alterar su copia. El código de la referencia es un resultado final, no una solución que el alumno deba pegar al inicio.

Criterio de aceptación: seguir los 24 pasos, compilar y ejecutar los tests; comprobar los resultados indicados y registrar las actividades manuales. Al finalizar, la solución debe reproducir el comportamiento descrito.

PASO 3.4.1 — Crear la carpeta del punto 3.4 a partir del punto 3.3

Cada punto del curso parte del contenido del punto anterior. El punto 3.4 se construye sobre el punto 3.3 copiando su contenido y añadiendo archivos nuevos sin modificar los existentes.

1. En el Explorador de archivos localiza `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.3-separacion-capas`. Comprueba que es el estado anterior completo, con los endpoints y tests ya construidos.
2. Crea una carpeta personal `3.4-implementacion-aspnet-core` bajo `M03_ARQUITECTURA_HEXAGONAL/PROYECTO`. Copia en ella el contenido del punto anterior, sin cambiarlo en su origen.
3. Desde la raíz de `3.4-implementacion-aspnet-core` comprueba que tienes `src/AceriaApp.Api`, `tests/AceriaApp.Api.Tests`, `AceriaApp.http` y los archivos de configuración.
4. No copies `bin/`, `obj/` ni credenciales. Para comparar puedes usar los proyectos de referencia; los nuevos archivos de este punto se introducirán solo cuando llegues a su paso.

Verificación concreta. al ejecutar `dir 3.4-implementacion-aspnet-core`, deben aparecer `AceriaApp.sln`, `README.md`, `src`, `tests` y `docs`.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

PASO 3.4.2 — Abrir la solución copiada en Visual Studio 2026

Abrir la solución copiada permite trabajar en el punto 3.4 sobre la base del punto 3.3. La compilación verifica que la copia es correcta.

1. Abre en Visual Studio la solución `AceriaApp.sln` de `3.4-implementacion-aspnet-core`; si la primera etapa aún no la tiene, crea una solución con `dotnet new sln -n AceriaApp`.
2. Desde la raíz del punto ejecuta `dotnet sln AceriaApp.sln add src/AceriaApp.Api/AceriaApp.Api.csproj tests/AceriaApp.Api.Tests/AceriaApp.Api.Tests.csproj` para la etapa inicial; en las siguientes etapas la solución viene heredada.
3. Verifica con `dotnet sln AceriaApp.sln list` que todas las rutas corresponden a proyectos presentes en la carpeta de esta etapa.
4. Ejecuta `dotnet build AceriaApp.sln -c Release` antes de tocar las clases y guarda su resultado como punto de comparación.

Verificación concreta. La solución abierta contiene los proyectos existentes de la etapa anterior y compila antes de los nuevos archivos.

Diagnóstico si falla.

- Abrir la solución del punto 3.3 en lugar de la del punto 3.4: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener la API del punto 3.3 en ejecución: el puerto queda ocupado y la compilación falla.

PASO 3.4.3 — Verificar que la carpeta `docs` contiene los documentos del punto 3.3

El punto 3.4 parte del contenido del punto 3.3, que ya incluye la carpeta `docs` con dieciséis documentos. No se modifica ningún documento existente.

1. Abre `docs` dentro de `3.4-implementacion-aspnet-core`. Si es la primera etapa y no existe, créala: botón derecho sobre la raíz del punto y `Agregar > Nueva carpeta`.
2. Comprueba que los documentos de la etapa anterior no se han modificado. A partir de aquí se añade una nueva decisión y un nuevo documento de diseño.
3. Anota los nombres de los documentos que recibes y los que crearás: `implementacion-aspnet-core.md` y `ADR-013-implementacion-aspnet-core.md`.
4. No reescribas un ADR anterior para explicar un diseño nuevo. Comprueba que los documentos heredados mantienen su contenido entre etapas.

Verificación concreta. la carpeta `docs` contiene los dieciséis documentos del punto 3.3 sin modificaciones.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 3.3.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 3.4.4 — Crear el documento de implementación en ASP.NET Core en `docs`

El documento de implementación en ASP.NET Core es el artefacto de `docs` específico del punto 3.4. Incluye el cierre del módulo 3 con el resumen del estado del proyecto. No modifica los dieciséis documentos existentes del punto 3.3.

1. En la carpeta `docs` de `3.4-implementacion-aspnet-core` crea `implementacion-aspnet-core.md` mediante `Agregar > Nuevo elemento` o el editor Markdown.
2. Copia el documento de este paso y estudia sus apartados. Relaciónalo con los bloques de teoría del punto 3.4; no debe limitarse a una lista de nombres.
3. Distingue los componentes del catálogo arquitectónico de las clases y los servicios implementados en el proyecto.
4. Guarda el Markdown, abre su vista previa y anota dos decisiones observables que podrás verificar posteriormente con los tests.

Archivo que debe quedar tras este paso: `docs/implementacion-aspnet-core.md`

```
# Implementación en ASP.NET Core — Punto 3.4

## ASP.NET Core como adaptador de entrada

Los endpoints traducen entre HTTP y los puertos de entrada. No contienen reglas de negocio.

## ASP.NET Core como contenedor de dependencias

El contenedor registra las implementaciones de los puertos. Inyecta las implementaciones en los endpoints.

## Ciclos de vida

- Transient: nueva instancia en cada resolución.
- Scoped: una instancia por ámbito de petición.
- Singleton: una instancia por aplicación.

## Registro de adaptadores

| Adaptador | Ciclo |
```

```
|-----------|-------|
| OrdenRepositorioSqlServer | Scoped |
| InventarioRepositorioSqlServer | Scoped |
| PublicadorEventosRabbitMq | Singleton |
| ClienteInventarioHttp | Scoped |

## Endpoints como adaptadores de entrada

Los endpoints se agrupan por recurso en clases de extensión.

## Repositorios como adaptadores de salida

Los repositorios implementan los puertos de salida y conocen EF Core.

## Configuración del arranque

El arranque se organiza con métodos de extensión por área de configuración.

## Pruebas de la configuración

Los tests verifican los registros y los ciclos de vida.

## Cierre del módulo 3

El proyecto AceriaApp dispone de:

- Organización por capas con regla de dependencia.
- Puertos y adaptadores en capas separadas.
- Separación física entre dominio, aplicación e infraestructura.
- Contenedor de dependencias configurado con métodos de extensión.
- Setenta y cuatro pruebas unitarias acumuladas en catorce proyectos de tests.
- Doce puntos completados (1.1 a 1.5, 2.1 a 2.4 y 3.1 a 3.4).

## Corrección de compatibilidad con el repositorio vigente

La implementación de partida no es `AceriaApp.Monolito.Api`, sino la Minimal API del proyecto
`M02_APIS_REST_ASPNET_CORE/PROYECTO/2.4-documentacion-openapi`. Los ejemplos de esta ficha describen
conceptos de diseño; su implementación real se incorpora al código en 3.3 y 3.4, con puertos de aplicación e
infraestructura SQL Server. No se considera registrado un adaptador solo por mencionarlo en un catálogo.
```

Verificación concreta. el archivo `docs\implementacion-aspnet-core.md` existe con las nueve secciones.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 3.4.5 — Crear la carpeta `Implementacion` en el proyecto `AceriaApp.Api`

La carpeta `Implementacion` albergará las clases de validación de la configuración del contenedor. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En `src/AceriaApp.Api` crea la carpeta `Implementacion` para las clases de validación de registros de servicios.
2. Añade también la carpeta `Hexagonal` y crea `HexagonalOrdenesEndpoints.cs` copiando el bloque completo de este paso.
3. Comprueba que los endpoints obtienen un `OrdenesCasoUso` por inyección y que no conocen `HexagonalDbContext` ni comandos EF Core.
4. No registres aún el grupo de rutas: primero se conectará el contenedor y después el arranque.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/Hexagonal/HexagonalOrdenesEndpoints.cs`

```
using AceriaApp.Hexagonal.Aplicacion.CasosUso;
using AceriaApp.Hexagonal.Dominio;
```

```
namespace AceriaApp.Api.Hexagonal;

public sealed record CrearOrdenHexagonalRequest(string Cliente, string Material, int Cantidad);
public sealed record OrdenHexagonalDto(Guid Id, string Cliente, string Material,
    int Cantidad, string Estado, DateTime FechaCreacion)
{
    public static OrdenHexagonalDto Desde(OrdenHexagonal orden) =>
        new(orden.Id, orden.Cliente, orden.Material, orden.Cantidad, orden.Estado, orden.FechaCreacion);
}

/// <summary>Adaptador HTTP: traduce peticiones en llamadas a los casos de uso.</summary>
public static class HexagonalOrdenesEndpoints
{
    public static IEndpointRouteBuilder MapHexagonalOrdenes(this IEndpointRouteBuilder app)
    {
        var group = app.MapGroup("/api/v1/hexagonal/ordenes").WithTags("Ordenes hexagonales");
        group.MapGet("", async (OrdenesCasoUso uso, CancellationToken ct) =>
        {
            var ordenes = await uso.ListarAsync(ct);
            return Results.Ok(ordenes.Select(OrdenHexagonalDto.Desde));
        }).WithName("ListarOrdenesHexagonales").Produces<IEnumerable<OrdenHexagonalDto>>(200);

        group.MapGet("/{id:guid}", async (Guid id, OrdenesCasoUso uso, CancellationToken ct) =>
        {
            var orden = await uso.ObtenerAsync(id, ct);
            IResult response = orden is null ? Results.NotFound() :
Results.Ok(OrdenHexagonalDto.Desde(orden));
            return response;
        }).WithName("ObtenerOrdenHexagonal").Produces<OrdenHexagonalDto>(200).Produces(404);

        group.MapPost("", async (CrearOrdenHexagonalRequest body, OrdenesCasoUso uso, CancellationToken ct)
=>
        {
            if (string.IsNullOrWhiteSpace(body.Cliente) || string.IsNullOrWhiteSpace(body.Material) ||
body.Cantidad <= 0)
                return Results.ValidationProblem(new Dictionary<string, string[]> { ["body"] = new[]
{ "Cliente, material y cantidad positiva son obligatorios." } });
            if (body.Cantidad > 500)
                return Results.Problem(title: "Capacidad del taller superada.", statusCode: 422);
            var orden = await uso.CrearAsync(body.Cliente, body.Material, body.Cantidad, ct);
            return Results.CreatedAtRoute("ObtenerOrdenHexagonal", new { id = orden.Id },
OrdenHexagonalDto.Desde(orden));
        }).WithName("CrearOrdenHexagonal").Produces<OrdenHexagonalDto>(201)
          .ProducesValidationProblem(400).ProducesProblem(422);

        group.MapPost("/{id:guid}/finalizar", async (Guid id, OrdenesCasoUso uso, CancellationToken ct) =>
        {
            try
            {
                var orden = await uso.FinalizarAsync(id, ct);
                IResult response = orden is null ? Results.NotFound() :
Results.Ok(OrdenHexagonalDto.Desde(orden));
            return response;
            }
            catch (InvalidOperationException)
            {
                return Results.Problem(title: "La orden ya está finalizada.", statusCode: 409);
            }
        }).WithName("FinalizarOrdenHexagonal").Produces<OrdenHexagonalDto>(200).Produces(404).ProducesProblem
(409);
        return app;
    }
}
```

Verificación concreta. la carpeta `Implementacion` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: viola la regla de no modificar archivos del punto anterior.

- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 3.4.6 — Crear la clase `RegistroServicio`

`RegistroServicio` describe una entrada de composición y sus propiedades. Declararla en una lista no equivale a registrar la implementación mediante `AddScoped` ni a resolver una dependencia real.

1. Crea `src/AceriaApp.Api/Implementacion/RegistroServicio.cs` y escribe el código íntegro mostrado en el bloque asociado al paso.
2. Comprueba que el namespace es `AceriaApp.Api.Implementacion`, correspondiente a la carpeta de este punto.
3. Lee cada propiedad y diferencia valores de configuración de objetos de dominio. No añadas dependencias SQL o ASP.NET Core a este modelo de catálogo.
4. Guarda y compila con `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`; si faltan tipos, revisa el namespace y los usings implícitos.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/Implementacion/RegistroServicio.cs`

```
namespace AceriaApp.Api.Implementacion;

public class RegistroServicio
{
    public string Puerto { get; init; } = string.Empty;
    public string Implementacion { get; init; } = string.Empty;
    public string Ciclo { get; init; } = string.Empty;
    public bool EsAdaptadorDeSalida { get; init; }
}
```

Verificación concreta. la clase `RegistroServicio` compila sin errores y expone cuatro propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 3.4.7 — Crear la clase `ValidadorRegistroServicios`

`ValidadorRegistroServicios` examina el catálogo docente. Debe comprobarse por separado que Program.cs registra el puerto, que el contenedor resuelve la implementación y que la petición HTTP la utiliza.

1. Abre o crea `src/AceriaApp.Api/Implementacion/ValidadorRegistroServicios.cs` según el bloque de C# de este paso.
2. Escribe la clase pública `ValidadorRegistroServicios` y el record de resultado; comprueba sus tipos de entrada y las ramas de error.
3. Recorre con el depurador un caso válido y otro inválido. Localiza dónde se agrega cada mensaje y qué condición determina el resultado final.
4. Guarda, compila y evita añadir el mismo validador en otro fichero. Las pruebas de esta regla se crearán en el paso 14.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/Implementacion/ValidadorRegistroServicios.cs`

```
namespace AceriaApp.Api.Implementacion;

public class ValidadorRegistroServicios
{
    private static readonly string[] CiclosPermitidos = { "Transient", "Scoped", "Singleton" };
```

```
    public ResultadoRegistro Validar(IEnumerable<RegistroServicio> registros)
    {
        var errores = new List<string>();
        var lista = registros.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay registros declarados.");
        }

        foreach (var registro in lista)
        {
            if (string.IsNullOrWhiteSpace(registro.Puerto))
            {
                errores.Add("Un registro no declara el puerto.");
            }

            if (string.IsNullOrWhiteSpace(registro.Implementacion))
            {
                errores.Add($"El puerto '{registro.Puerto}' no declara implementación.");
            }

            if (!CiclosPermitidos.Contains(registro.Ciclo))
            {
                errores.Add($"Ciclo no permitido en el puerto '{registro.Puerto}': '{registro.Ciclo}'.");
            }

            if (registro.EsAdaptadorDeSalida && registro.Ciclo == "Transient")
            {
                errores.Add($"Un adaptador de salida no debería registrarse como Transient:
'{registro.Puerto}'.");
            }
        }

        return new ResultadoRegistro(errores.Count == 0, errores);
    }
}

public record ResultadoRegistro(bool EsValido, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorRegistroServicios` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<string>` y `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` y `ToList` no se resuelven.
- No comprobar el caso de lista vacía: el validador devuelve válido sin registros.

PASO 3.4.8 — Añadir los `using` necesarios en `ValidadorRegistroServicios`

Los tipos `List<string>` y `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. Los métodos `Contains` y `ToList` pertenecen a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. Abre `src/AceriaApp.Api/Program.cs` y localiza los `using` de la plantilla M02.
2. Añade los tres imports de `AceriaApp.Api.Hexagonal`, `AceriaApp.Hexagonal.Infraestructura` y `AceriaApp.Hexagonal.Infraestructura.Datos` mostrados.
3. Comprueba que se mantienen los registros FluentValidation y Swagger existentes y que no se ha duplicado `CreateBuilder`.
4. Guarda y compila. Si un namespace no se reconoce, revisa los `ProjectReference` añadidos en 3.3.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/Program.cs` (modificación; buscar `var builder=WebApplication.CreateBuilder(args);`)

```
using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Infraestructura;
using AceriaApp.Hexagonal.Infraestructura.Datos;
```

Verificación concreta. `ValidadorRegistroServicios.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `Contains` y `ToList` no se resuelven.
- Añadir los usings después del namespace: el compilador no los reconoce.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 3.4.9 — Crear la clase `CatalogoRegistros`

`CatalogoRegistros` clasifica servicios según las reglas del ejercicio. El recorrido funcional se acredita únicamente cuando el host y sus adaptadores trabajan con la base real.

1. En `Program.cs` localiza `var app=builder.Build();` y, justo antes, registra `builder.Services.AddServiciosHexagonales(builder.Configuration);`.
2. Comprueba el constructor de `OrdenesCasoUso`, que solicita `IOrdenRepositorio`, y la clase concreta `OrdenRepositorioSqlServer` registrada como Scoped.
3. No registres la interfaz directamente contra un nuevo objeto con `new`: deja que DI resuelva DbContext y repositorio por petición.
4. Guarda y compila. En el paso de pruebas comprobarás que el host HTTP puede resolver el puerto de repositorio.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/Implementacion/CatalogoRegistros.cs`

```
namespace AceriaApp.Api.Implementacion;

public class CatalogoRegistros
{
    public static List<RegistroServicio> RegistrosPorDefecto()
    {
        return new List<RegistroServicio>
        {
            new() { Puerto = "IOrdenRepositorio", Implementacion = "OrdenRepositorioSqlServer", Ciclo =
"Scoped", EsAdaptadorDeSalida = true },
            new() { Puerto = "IInventarioRepositorio", Implementacion = "InventarioRepositorioSqlServer",
Ciclo = "Scoped", EsAdaptadorDeSalida = true },
            new() { Puerto = "IPublicadorEventos", Implementacion = "PublicadorEventosRabbitMq", Ciclo =
"Singleton", EsAdaptadorDeSalida = true },
            new() { Puerto = "IOrdenServicio", Implementacion = "OrdenServicio", Ciclo = "Scoped",
EsAdaptadorDeSalida = false },
            new() { Puerto = "IInventarioServicio", Implementacion = "InventarioServicio", Ciclo = "Scoped",
EsAdaptadorDeSalida = false }
        };
    }
}
```

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/Program.cs` (modificación; buscar `var app=builder.Build();`)

```
builder.Services.AddServiciosHexagonales(builder.Configuration);
```

Verificación concreta. la clase `CatalogoRegistros` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.

- Devolver `List<T>` mutable: expone la implementación interna.

PASO 3.4.10 — Añadir el `using` necesario en `CatalogoRegistros`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. En `Program.cs`, antes de `app.Run();`, añade el bloque de inicialización del contexto hexagonal mostrado.
2. Esta inicialización se ejecuta solo en Development y cuando `SkipDatabaseInitialization` no está activo.
3. Comprueba que `HexagonalDbContext` utiliza su propia conexión de base de datos, separada de la inicialización M02.
4. Conserva la inicialización de EF Core heredada de M02 y el flujo de tests SQL Server con base exclusiva; no borres rutas existentes.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/Program.cs` (modificación; buscar `app.Run();`)

```
// El contexto hexagonal usa su propia base para que EnsureCreated no dependa de
// las tablas heredadas del curso M02.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<HexagonalDbContext>()
        .Database.EnsureCreatedAsync();
}
```

Verificación concreta. La inicialización del nuevo DbContext solo sucede en Development y no depende de la base M02.

Diagnóstico si falla.

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 3.4.11 — Crear el proyecto de tests del punto 3.4

El punto 3.4 añade un proyecto de tests específico para las clases de implementación. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

1. Dentro de `tests` crea el proyecto xUnit `AceriaApp.Implementacion.Tests` con destino `net8.0`. No modifiques ni sustituyas `AceriaApp.Api.Tests`, que contiene las pruebas HTTP heredadas de M02.
2. Crea el archivo `tests/AceriaApp.Implementacion.Tests/AceriaApp.Implementacion.Tests.csproj` y escribe el XML completo del bloque del laboratorio, con xUnit y FluentAssertions 6.12.2.
3. Comprueba que incluye un `ProjectReference` hacia `../../src/AceriaApp.Api/AceriaApp.Api.csproj`; agrega el nuevo proyecto con `dotnet sln AceriaApp.sln add tests/AceriaApp.Implementacion.Tests/AceriaApp.Implementacion.Tests.csproj`.
4. Ejecuta `dotnet restore AceriaApp.sln` y comprueba que no se ha generado un test `UnitTest1` no previsto.

Archivo que debe quedar tras este paso: `tests/AceriaApp.Implementacion.Tests/AceriaApp.Implementacion.Tests.csproj`

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsTestProject>true</IsTestProject><IsPackable>false</IsPackable></PropertyGroup>
  <ItemGroup>
```

```
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El proyecto de tests `AceriaApp.Implementacion.Tests` restaura, referencia a `AceriaApp.Api` y no sustituye ninguna suite heredada.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 3.3: viola la regla de no modificar archivos del punto anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 3.4.12 — Añadir la referencia del proyecto de tests a la API `AceriaApp.Api`

La referencia permite que el proyecto de tests acceda a las clases de `Implementacion` del proyecto `AceriaApp.Api`.

1. Busca `app.Run();` en `Program.cs` e inserta `app.MapHexagonalOrdenes();` inmediatamente antes.
2. Comprueba que el método está implementado como extensión de `IEndpointRouteBuilder` y que registra GET colección, GET detalle, POST y finalizar.
3. No dupliques `/api/v1/ordenes`: el nuevo adaptador pedagógico usa `/api/v1/hexagonal/ordenes` para evitar romper el contrato ya aprobado de M02.
4. Guarda, compila y verifica que Swagger descubre las operaciones nuevas cuando la API corre en Development.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/Program.cs` (modificación; buscar `app.Run();`)

```
app.MapHexagonalOrdenes();
```

Verificación concreta. GET/POST/finalizar están registrados por el adaptador HTTP y utilizan `OrdenesCasoUso`.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 3.4.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Abre `src/AceriaApp.Api/appsettings.json` y conserva la conexión existente `AceriaApp` utilizada por la API de M02.
2. Añade `ConnectionStrings:AceriaHexagonal` con LocalDB en la base `AceriaAppM03Hexagonal` como muestra el JSON completo del bloque.
3. Valida la sintaxis del JSON, especialmente la doble barra invertida `\` en el nombre `(localdb)\MSSQLLocalDB`.

4. Arranca en Development y comprueba que los dos contextos trabajan con bases independientes; los tests también usan SQL Server real para ambos, con dos bases exclusivas distintas.

Archivo que debe quedar tras este paso: `src/AceriaApp.Api/appsettings.json`

```
{
  "ConnectionStrings": {
    "AceriaApp": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaAppM02;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaHexagonal": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaAppM03Hexagonal;Trusted_Connection=True;TrustServerCertificate=True"
  }
}
```

Verificación concreta. Existen las dos cadenas de conexión de manera independiente: AceriaApp y AceriaHexagonal.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 3.4.14 — Crear el archivo `ValidadorRegistroServiciosTests.cs`

Los tests de registro ejercitan condiciones del catálogo. No sustituyen los ensayos de integración que resuelven servicios en ASP.NET Core y recorren una operación con SQL Server.

1. Crea `tests/AceriaApp.Implementacion.Tests/ValidadorRegistroServiciosTests.cs` y escribe completo el primer bloque de pruebas de este punto.
2. Revisa el `using AceriaApp.Api.Implementacion` para que coincida con el namespace actualizado y conserva `using Xunit;` y `using FluentAssertions;`.
3. Localiza los casos correctos e incorrectos del validador y comprueba que las aserciones describen resultados de código ejecutable.
4. Guarda, compila y ejecuta el test específico. No cambies las expectativas para ocultar una regla equivocada.

Archivo que debe quedar tras este paso: `tests/AceriaApp.Implementacion.Tests/ValidadorRegistroServiciosTests.cs`

```
using AceriaApp.Api.Implementacion;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Implementacion.Tests;

public class ValidadorRegistroServiciosTests
{
    private readonly ValidadorRegistroServicios _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoRegistros.RegistrosPorDefecto());

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinRegistros_DevuelveError()
    {
        var resultado = _validador.Validar(new List<RegistroServicio>());
```

```
        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay registros"));
    }

    [Fact]
    public void Validar_CicloNoPermitido_DevuelveError()
    {
        var registros = new List<RegistroServicio>
        {
            new() { Puerto = "IPuertoX", Implementacion = "ImplementacionX", Ciclo = "CicloInvalido" }
        };

        var resultado = _validador.Validar(registros);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Ciclo no permitido"));
    }

    [Fact]
    public void Validar_AdaptadorDeSalidaTransient_DevuelveError()
    {
        var registros = new List<RegistroServicio>
        {
            new() { Puerto = "IOrdenRepositorio", Implementacion = "OrdenRepositorioSqlServer", Ciclo =
"Transient", EsAdaptadorDeSalida = true }
        };

        var resultado = _validador.Validar(registros);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Transient"));
    }

    [Fact]
    public void Validar_SinImplementacion_DevuelveError()
    {
        var registros = new List<RegistroServicio>
        {
            new() { Puerto = "IPuertoX", Implementacion = string.Empty, Ciclo = "Scoped" }
        };

        var resultado = _validador.Validar(registros);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara implementación"));
    }
}
```

Verificación concreta. Los tests de `AceriaApp.Implementacion.Tests` compilan y ejecutan las aserciones propuestas para `Implementacion`.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Implementacion;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de lista vacía: el mensaje de error no se verifica.

PASO 3.4.15 — Crear el archivo `CatalogoRegistrosTests.cs`

`CatalogoRegistrosTests` confirma reglas de clasificación, no la existencia de una instancia de infraestructura activa. Es importante identificar qué problema demuestra cada prueba.

1. Crea `tests/AceriaApp.Implementacion.Tests/CatalogoRegistrosTests.cs` y escribe el segundo bloque de pruebas del catálogo.
2. Comprueba que las llamadas a los métodos de ejemplo de `{catalog}` son compatibles con las propiedades que realmente expone el código.

3. Guarda y ejecuta `dotnet test tests/AceriaApp.Implementacion.Tests/AceriaApp.Implementacion.Tests.csproj -c Release`, de forma que se ejecuten todas las pruebas nuevas.
4. Verifica que las aserciones prueban los valores declarados, pero no confunde un catálogo didáctico con un registro en tiempo de ejecución.

Archivo que debe quedar tras este paso: `tests/AceriaApp.Implementacion.Tests/CatalogoRegistrosTests.cs`

```
using AceriaApp.Api.Implementacion;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Implementacion.Tests;

public class CatalogoRegistrosTests
{
    [Fact]
    public void RegistrosPorDefecto_ContieneCincoRegistros()
    {
        var registros = CatalogoRegistros.RegistrosPorDefecto();

        registros.Should().HaveCount(5);
    }

    [Fact]
    public void RegistrosPorDefecto_ContieneTresAdaptadoresDeSalidaYDosPuertosDeEntrada()
    {
        var registros = CatalogoRegistros.RegistrosPorDefecto();

        registros.Count(r => r.EsAdaptadorDeSalida).Should().Be(3);
        registros.Count(r => !r.EsAdaptadorDeSalida).Should().Be(2);
    }

    [Fact]
    public void RegistrosPorDefecto_TodosLosPuertosTienenImplementacion()
    {
        var registros = CatalogoRegistros.RegistrosPorDefecto();

        registros.Should().OnlyContain(r => !string.IsNullOrWhiteSpace(r.Implementacion));
    }
}
```

Verificación concreta. Los tests de `AceriaApp.Implementacion.Tests` compilan y ejecutan las aserciones propuestas para `Implementacion`.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Implementacion;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Count` y `OnlyContain` no se resuelven.
- No verificar la distribución de adaptadores: el catálogo podría estar mal balanceado.

PASO 3.4.16 — Compilar la solución completa

La compilación de la solución comprueba las referencias y los contratos de tipos. Sigue siendo necesario verificar el caso de uso en HTTP y persistencia para demostrar que las piezas están conectadas.

1. Abre la terminal junto a `AceriaApp.sln` y ejecuta `dotnet build AceriaApp.sln -c Release` después de introducir todos los archivos del punto.
2. Comprueba que compilan la API de M02, el proyecto nuevo de tests y todos los proyectos de etapas anteriores.
3. Si aparece un error, identifica el primer archivo y la primera línea antes de tocar otra clase. Revisa referencias y namespaces.
4. Registra la salida de la compilación y no avances hasta obtener cero errores.

Verificación concreta. La solución completa compila en Release con cero errores. La capa de dominio no referencia EF Core ni ASP.NET Core.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 3.4.17 — Ejecutar todos los tests del punto 3.4

Ejecutar los tests del punto 3.4 verifica que las clases de implementación funcionan correctamente.

1. En `tests/AceriaApp.Api.Tests` crea `HexagonalFactory.cs` y `HexagonalHttpTests.cs` con los dos fragmentos de código.
2. La factoría conserva SQL Server en los dos DbContext y asigna una base exclusiva a cada contexto. `CreateHost` crea ambos esquemas con `EnsureCreated`; la limpieza elimina únicamente esas dos bases.
3. Ejecuta las cuatro pruebas HTTP: GET vacía, creación y consulta, errores 400/422, y finalización repetida 200/409.
4. Comprueba que las pruebas de los catálogos siguen pasando junto a los tests nuevos: la validación ya no es solo nominal.

Archivo que debe quedar tras este paso: `tests/AceriaApp.Api.Tests/HexagonalFactory.cs`

```
using AceriaApp.Api.Datos;
using AceriaApp.Hexagonal.Infraestructura.Datos;
using AceriaApp.PruebasSqlServer;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
using Microsoft.Extensions.Hosting;

namespace AceriaApp.Api.Tests;

public sealed class HexagonalFactory : WebApplicationFactory<Program>
{
    private readonly BaseSqlServer _legacy = BaseSqlServer.Crear();
    private readonly BaseSqlServer _hexagonal;
    public HexagonalFactory()
    {
        try { _hexagonal = BaseSqlServer.Crear(); }
        catch { _legacy.Dispose(); throw; }
    }
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Development");
        builder.UseSetting("SkipDatabaseInitialization", "true");
        builder.ConfigureTestServices(services =>
        {
            services.RemoveAll<DbContextOptions<AceriaDbContext>>();
            services.AddDbContext<AceriaDbContext>(o => o.UseSqlServer(_legacy.ConnectionString));
            services.RemoveAll<DbContextOptions<HexagonalDbContext>>();
            services.AddDbContext<HexagonalDbContext>(o => o.UseSqlServer(_hexagonal.ConnectionString));
        });
    }
    protected override IHost CreateHost(IHostBuilder builder)
    {
        var host = base.CreateHost(builder);
        using var scope = host.Services.CreateScope();
        // Cada modelo crea sus tablas en una base distinta del mismo motor SQL Server.
        scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreated();
        scope.ServiceProvider.GetRequiredService<HexagonalDbContext>().Database.EnsureCreated();
        return host;
```

```
    }
    protected override void Dispose(bool disposing)
    {
        try { base.Dispose(disposing); }
        finally
        {
            if (disposing)
            {
                try { _hexagonal.Dispose(); }
                finally { _legacy.Dispose(); }
            }
        }
    }
    public override async ValueTask DisposeAsync()
    {
        try { await base.DisposeAsync(); }
        finally
        {
            try { await _hexagonal.DisposeAsync(); }
            finally { await _legacy.DisposeAsync(); }
        }
    }
}
```

Archivo que debe quedar tras este paso: `tests/AceriaApp.Api.Tests/HexagonalHttpTests.cs`

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Hexagonal;
using AceriaApp.Api.Datos;
using AceriaApp.Hexagonal.Infraestructura.Datos;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Xunit;

namespace AceriaApp.Api.Tests;

public class HexagonalHttpTests
{
    [Fact]
    public async Task Get_ColeccionVacia_Es200()
    {
        using var factory = new HexagonalFactory();
        var response = await factory.CreateClient().GetAsync("/api/v1/hexagonal/ordenes");
        Assert.Equal(HttpStatusCode.OK, response.StatusCode);
    }

    [Fact]
    public async Task Post_CreaYGetConsultaPorPuerto()
    {
        using var factory = new HexagonalFactory();
        var client = factory.CreateClient();
        using var scope = factory.Services.CreateScope();
        var legacy = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        var hexagonal = scope.ServiceProvider.GetRequiredService<HexagonalDbContext>();
        Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", legacy.Database.ProviderName);
        Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", hexagonal.Database.ProviderName);
        Assert.NotEqual(legacy.Database.GetDbConnection().Database,
hexagonal.Database.GetDbConnection().Database);
        var created = await client.PostAsJsonAsync("/api/v1/hexagonal/ordenes",
            new CrearOrdenHexagonalRequest("ACX-001", "CHAPA", 10));
        Assert.Equal(HttpStatusCode.Created, created.StatusCode);
        Assert.NotNull(created.Headers.Location);
        var retrieved = await client.GetAsync(created.Headers.Location);
        Assert.Equal(HttpStatusCode.OK, retrieved.StatusCode);
        var dto = await retrieved.Content.ReadFromJsonAsync<OrdenHexagonalDto>();
        Assert.Equal("ACX-001", dto?.Cliente);
        Assert.Equal(10, dto?.Cantidad);
    }

    [Fact]
    public async Task Post_ValidacionYCapacidadRespetan400Y422()
    {
        using var factory = new HexagonalFactory();
```

```
        var client = factory.CreateClient();
        var invalid = await client.PostAsJsonAsync("/api/v1/hexagonal/ordenes",
            new CrearOrdenHexagonalRequest("", "CHAPA", 10));
        Assert.Equal(HttpStatusCode.BadRequest, invalid.StatusCode);
        var excess = await client.PostAsJsonAsync("/api/v1/hexagonal/ordenes",
            new CrearOrdenHexagonalRequest("ACX", "CHAPA", 501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity, excess.StatusCode);
    }

    [Fact]
    public async Task Finalizar_DosVeces_Responde200Y409()
    {
        using var factory = new HexagonalFactory();
        var client = factory.CreateClient();
        var created = await client.PostAsJsonAsync("/api/v1/hexagonal/ordenes",
            new CrearOrdenHexagonalRequest("ACX", "CHAPA", 5));
        var orden = await created.Content.ReadFromJsonAsync<OrdenHexagonalDto>();
        Assert.NotNull(orden);
        var url = $"/api/v1/hexagonal/ordenes/{orden.Id}/finalizar";
        Assert.Equal(HttpStatusCode.OK, (await client.PostAsync(url, null)).StatusCode);
        Assert.Equal(HttpStatusCode.Conflict, (await client.PostAsync(url, null)).StatusCode);
    }
}
```

Lectura del aislamiento. `_legacy` y `_hexagonal` llaman por separado a `BaseSqlServer.Crear()`: sus GUID impiden compartir tablas aunque los contextos se ejecuten en el mismo servidor. `SkipDatabaseInitialization` evita tocar las bases habituales de la aplicación. La factoría inicializa expresamente ambos esquemas tras crear el host y cierra el host antes de eliminar las dos bases. La prueba `Post_CreaYGetConsultaPorPuerto` verifica `ProviderName` en ambos contextos y que los nombres de base son distintos antes de recorrer POST → GET por el puerto de aplicación. El helper, el paquete de Testcontainers y su enlace `Compile` ya existen desde M02 2.1.22; no crees una segunda copia del helper dentro del proyecto xUnit.

Verificación concreta. Las cuatro pruebas HTTP hexagonales superan GET, POST→GET, 400/422 y finalizar 200/409 usando SQL Server real con bases separadas y comprobaciones del proveedor.

Diagnóstico si falla.

- Ejecutar `dotnet test` sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 3.4.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 3.4 pasan conjuntamente.

1. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después de compilar toda la solución en Release.
2. Comprueba que pasan los tests de M02 y todos los nuevos del módulo M03 acumulados hasta este punto.
3. Si falla alguna suite previa, vuelve a comprobar si el último cambio alteró una dependencia o una configuración del servidor.
4. Registra las pruebas totales, fallidas y omitidas y compáralas con el resultado previsto para esta etapa.

Verificación concreta. Pruebas acumuladas previstas para 3.4: 50, 0 fallidas. Comprueba estos valores en la salida de `dotnet test`.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

PASO 3.4.19 — Crear el README del punto 3.4

El README del punto 3.4 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 3.3.

1. Abre `README.md` en la raíz del punto `3.4-implementacion-aspnet-core` y sustitúyelo por el README específico del bloque de este paso.
2. Comprueba que los nombres de proyectos y las pruebas acumuladas coinciden con la solución y con la salida de `dotnet test` de esta etapa.
3. Revisa la ubicación del documento `docs` y los comandos `dotnet build` y `dotnet test` que utilizará el alumno.
4. Guarda la versión del punto actual sin sobrescribir el README de la etapa anterior.

Archivo que debe quedar tras este paso: `README.md`

```
# Punto 3.4 — Carpeta del punto 3.4 a partir del punto 3.3

**Curso:** Microservicios y arquitectura hexagonal con ASP.NET Core 8
**Código de referencia:** AceriaApp, en continuidad con M02 2.4.

## Objetivo

el documento de implementación en ASP.NET Core es el artefacto de `docs` específico del punto 3.4. Incluye el cierre del módulo 3 con el resumen del estado del proyecto. No modifica los dieciséis documentos existentes del punto 3.3.

## Estado verificado del punto

- 9 proyectos en la solución `AceriaApp.sln` al finalizar este punto.
- 50 pruebas xUnit acumuladas previstas (incluyendo las 16 de M02 y las nuevas del módulo 3).
- Clases del catálogo `Implementacion` en `src/AceriaApp.Api/Implementacion`.
- Documentación `docs/implementacion-aspnet-core.md` y decisión `docs/ADR-013-implementacion-aspnet-core.md`.
- Los endpoints REST del módulo 2 se conservan sin cambios funcionales.

## Cómo ejecutarlo

1. Abrir el archivo `AceriaApp.sln` de esta carpeta en Visual Studio, con SDK .NET 8.
2. Ejecutar `dotnet restore AceriaApp.sln` y después `dotnet build AceriaApp.sln -c Release`.
3. Ejecutar `dotnet test AceriaApp.sln -c Release --no-build`. La suite M03 se construye por adición desde M02.
4. La aplicación principal utiliza SQL Server Express LocalDB; los tests HTTP mantienen SQL Server y crean bases exclusivas independientes mediante `BaseSqlServer.Crear()`.
5. En 3.4, los endpoints hexagonales reales están en `/api/v1/hexagonal/ordenes` y utilizan un repositorio EF Core a través de un puerto de aplicación.

## Qué se ha añadido

la clase `RegistroServicio` modela un registro del contenedor de dependencias con el puerto, la implementación, el ciclo y si es adaptador de salida. Es la unidad de información del validador.
```

Verificación concreta. `README.md` describe correctamente los 9 proyectos y 50 pruebas acumuladas del curso vigente.

Diagnóstico si falla.

- Modificar el README del punto 3.3: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 3.3: rompe la separación.

PASO 3.4.20 — Crear el ADR-013 del punto 3.4

El ADR-013 documenta la decisión de implementar la arquitectura hexagonal en ASP.NET Core y de cerrar el módulo 3. Es un artefacto nuevo del punto 3.4 que no modifica los ADR existentes.

1. En la carpeta `docs` crea `ADR-013-implementacion-aspnet-core.md` y copia el documento de decisión del bloque del laboratorio.
2. Lee Estado, Contexto, Decisión, Alternativas y Consecuencias y relaciona el ADR con el código nuevo de la etapa.
3. Explica por qué no basta con declarar nombres de capas si el código puede saltarse la interfaz o registrar una dependencia incorrecta.
4. Comprueba que los ADR anteriores están presentes y no han sido sobrescritos.

Archivo que debe quedar tras este paso: `docs/ADR-013-implementacion-aspnet-core.md`

```
# ADR-013: Implementación de la arquitectura hexagonal en ASP.NET Core

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado la separación física entre dominio, aplicación e infraestructura. Es
necesario definir cómo se implementa la arquitectura hexagonal en ASP.NET Core y cómo se configura el
contenedor de dependencias.

## Decisión
Los endpoints se agrupan por recurso en clases de extensión. Los repositorios se implementan como adaptadores
de salida en la capa de infraestructura. Los servicios de aplicación se implementan como puertos de entrada.
El contenedor de dependencias se configura con métodos de extensión por área. La configuración se valida con
tests que verifican los registros y los ciclos de vida. Se adopta el ciclo Scoped para los adaptadores de
base de datos y Singleton para los adaptadores sin estado.

## Alternativas consideradas
- Registro directo en Program.cs: descartado por crecimiento del archivo.
- Uso de un contenedor externo: descartado por simplicidad del proyecto.
- Ciclo Transient para todos los servicios: descartado por coste de instanciación.

## Consecuencias
- Positivas: configuración legible, validación automática, ciclos adecuados.
- Negativas: la configuración requiere disciplina en el registro.
- Mitigación: validación automática con el validador.

## Aplicación en AceriaApp

Se parte de la API real y probada de M02, se conservan sus endpoints y se añaden reglas de dependencia
verificadas en código. Los proyectos de Dominio, Aplicación e Infraestructura deben obedecer la dirección de
referencias; en 3.4 se expone un nuevo adaptador HTTP funcional. El catálogo describe las relaciones
arquitectónicas; cada implementación se comprueba en su proyecto correspondiente.
```

Verificación concreta. el archivo `docs\ADR-013-implementacion-aspnet-core.md` existe con las cinco secciones.

Diagnóstico si falla.

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-013`: rompe la convención.

PASO 3.4.21 — Verificar que no se ha modificado ningún archivo del punto 3.3

La comparación con `fc` verifica que todos los documentos del punto 3.3 no se han modificado.

1. Localiza el mismo documento heredado tanto en `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.3-separacion-capas` como en `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.4-implementacion-aspnet-core`.

2. Compara sus hashes SHA-256 o usa un diff y confirma que no se reescribió al introducir el nuevo punto.
3. Distingue de él los archivos añadidos en este punto: catálogo, validador, pruebas y documentación nueva.
4. Documenta cualquier diferencia inesperada; el nuevo código no exige alterar los archivos de etapas anteriores.

Verificación concreta. Los archivos heredados de `docs` y los endpoints M02 no han sido sobrescritos.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Olvidar el flag `/L`: `fc` compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 3.4.22 — Verificar la estructura del punto 3.4

La verificación estructural confirma que el punto 3.4 añade archivos nuevos sin modificar los existentes.

1. Desde la raíz de `M03_ARQUITECTURA_HEXAGONAL/PROYECTO` ejecuta `tree /F 3.4-implementacion-aspnet-core` en Windows.
2. Localiza `src/AceriaApp.Api/Implementacion`, `tests/AceriaApp.Implementacion.Tests`, `README.md` y `docs/implementacion-aspnet-core.md`.
3. Comprueba los proyectos adicionales de dominio, aplicación e infraestructura si ya fueron introducidos por el punto 3.3.
4. Distingue los archivos del proyecto de los temporales `obj/` y `bin/`; ambos deben quedar excluidos de la entrega.

Verificación concreta. Los archivos que aparecen en el árbol corresponden a los fragmentos creados hasta este paso, y no hay código pendiente en carpetas fuera de la solución.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 3.3: viola la regla de no modificar.
- Omitir la carpeta `Implementacion`: los tests no encuentran las clases.

PASO 3.4.23 — Verificar el `.gitignore` del punto 3.4

El `.gitignore` se hereda del punto 3.3 sin modificaciones. La verificación confirma que no se ha tocado.

1. Abre el `.gitignore` de la etapa; si falta, crea uno que excluya `.vs/`, `bin/`, `obj/`, archivos personales y resultados de prueba.
2. Ejecuta `git status --short` en tu copia de trabajo y comprueba que no aparecen archivos binarios ni credenciales.
3. No ignores `src`, `tests`, `docs` ni las soluciones `.sln`, pues son parte del entregable reproducible.
4. Conserva el `.gitignore` heredado sin borrarlo ni modificar el de etapas anteriores.

Verificación concreta. No se incluyen `bin`, `obj`, contraseñas ni resultados de pruebas en el estado Git.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 3.3: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 3.4.24 — Hacer el commit del punto 3.4 y verificar el estado final del módulo 3

El commit del punto 3.4 cierra el módulo 3. Los commits de cada punto permiten comparar el estado del proyecto entre puntos consecutivos y entre módulos completos.

1. Antes de terminar 3.4, vuelve a ejecutar `dotnet build AceriaApp.sln -c Release` y `dotnet test AceriaApp.sln -c Release --no-build`.
2. Lee el reto de ampliación incluido tras los pasos obligatorios; si lo realizas, utiliza una copia aparte y deja los tests base intactos.
3. Si tu copia está bajo Git, ejecuta `git status`, añade solo lo trabajado y crea un commit descriptivo, sin modificar la rama canónica del profesor.
4. Conserva el resumen de pruebas y compara el proyecto final con la carpeta de referencia del punto; ambas deben representar el mismo código.

Verificación concreta. El punto termina con compilación y suite de pruebas completa, sin modificar la referencia de la etapa anterior.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Esta ampliación añade una operación de resumen al validador y una prueba de su resultado. **Realízala después de superar la etapa básica y sus tests.** Incorpora cada fragmento en la clase indicada y conserva el namespace del proyecto en el que trabajas.

Reto: añadir un método `Resumen` a `ValidadorRegistroServicios` que devuelva el número total de registros, el número de adaptadores de salida y el número de puertos de entrada, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorRegistroServicios.cs`, añadir el siguiente `record` al final del archivo:

```
public record ResumenRegistros(int Total, int AdaptadoresSalida, int PuertosEntrada);
```

Paso 2: Explicar línea por línea:

- **Línea 1:** `public record ResumenRegistros(` → declara un contrato inmutable.
- **Línea 2:** `int Total,` → número total de registros.
- **Línea 3:** `int AdaptadoresSalida,` → número de adaptadores de salida.
- **Línea 4:** `int PuertosEntrada);` → número de puertos de entrada.

Paso 3: En `ValidadorRegistroServicios.cs`, añadir el siguiente método estático dentro de la clase:

```
public static ResumenRegistros Resumen(IEnumerable<RegistroServicio> registros)
{
    var lista = registros.ToList();
    var adaptadores = lista.Count(r => r.EsAdaptadorDeSalida);
    var entradas = lista.Count - adaptadores;
    return new ResumenRegistros(lista.Count, adaptadores, entradas);
}
```

Paso 4: Explicar línea por línea:

- **Línea 1:** `public static ResumenRegistros Resumen(IEnumerable<RegistroServicio> registros)` → declara el método.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `var lista = registros.ToList();` → materializa la lista.
- **Línea 4:** `var adaptadores = lista.Count(r => r.EsAdaptadorDeSalida);` → cuenta los adaptadores.
- **Línea 5:** `var entradas = lista.Count - adaptadores;` → calcula los puertos de entrada por diferencia.
- **Línea 6:** `return new ResumenRegistros(lista.Count, adaptadores, entradas);` → devuelve el resumen.
- **Línea 7:** `}` → cierra el cuerpo.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorRegistroServiciosTests.cs`, añadir el siguiente test:

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CuentaCorrectamente()
{
    var resumen = ValidadorRegistroServicios.Resumen(CatalogoRegistros.RegistrosPorDefecto());

    resumen.Total.Should().Be(5);
    resumen.AdaptadoresSalida.Should().Be(3);
    resumen.PuertosEntrada.Should().Be(2);
}
```

Paso 7: Explicar línea por línea:

- **Línea 1:** `[Fact]` → marca el método como test de xUnit.
- **Línea 2:** `public void Resumen_ConCatalogoPorDefecto_CuentaCorrectamente()` → declara el test.
- **Línea 3:** `{` → abre el cuerpo.
- **Línea 4:** `var resumen = ValidadorRegistroServicios.Resumen(CatalogoRegistros.RegistrosPorDefecto());` → invoca el método.
- **Línea 5:** `resumen.Total.Should().Be(5);` → verifica el total.
- **Línea 6:** `resumen.AdaptadoresSalida.Should().Be(3);` → verifica los adaptadores.
- **Línea 7:** `resumen.PuertosEntrada.Should().Be(2);` → verifica los puertos de entrada.
- **Línea 8:** `}` → cierra el cuerpo.

Paso 8: Ejecutar `dotnet test tests\AceriaApp.Implementacion.Tests\AceriaApp.Implementacion.Tests.csproj` y verificar que pasan los nueve tests.

Paso 9: Verificar que el método `Resumen` no modifica ningún archivo del punto 3.3.

Paso 10: Hacer commit con `git commit -m "Reto 3.4: método Resumen en ValidadorRegistroServicios"`.

Verificación: la salida de `dotnet test` muestra `Passed: 9`.

Resultado verificable y cierre del punto 3.4

Código final de referencia: `PROYECTO/3.4-implementacion-aspnet-core/`. El contenido que debe crear el alumno está especificado por los bloques con su ruta exacta y el tipo de modificación. Comprueba el resultado compilando y ejecutando las pruebas de la etapa.

Continuidad: las dependencias existentes y la documentación heredada se conservan; el avance del punto 3.4 prepara el siguiente sin reescribir la etapa anterior.
